Verification of FTAD: A Financial Transaction Anomaly Detector

Bongeni Hashcat

BIST4110 Software Testing and Reliability\6 November 2026

Contents

Abstract 4

Introduction 5

Chapter 1 — Project Audit 6

1.1 Project Overview 6

1.2 Feature Inventory 6

1.3 Architecture Observations 7

1.4 SLOC Results 7

1.5 Existing Testing Infrastructure 8

1.6 Gaps and Compliance 8

Chapter 2 — Literature Review and Background Research 9

2.1 Introduction 9

2.2 Software Testing Fundamentals 10

2.3 Software Quality Assurance 10

2.4 Testing Methodologies and Techniques 11

2.5 Testing Levels 12

2.6 Testing Tools and Technologies 12

2.7 Performance Testing 13

2.8 Security Testing 13

2.9 Software Reliability 14

2.10 Related Studies and Existing Solutions 15

2.11 Literature Review Summary and Identified Gaps 16

2.12 Reference quality and verification 16

Chapter 3 — System Architecture 17

3.1 Overview 17

3.2 Layering 18
3.3 Module Responsibilities 18
3.4 Architectural Views 18
3.5 Data Flow and Storage 19
3.6 Error Handling 19

Chapter 4 — Testing Methodology and Test Plan 20
4.1 Introduction 20
4.2 Objectives and Strategy 21
4.3 Testing Levels 22
4.4 Test Design Techniques 23
4.5 Tools, Environment and Scope 23
4.6 Plan, Entry and Exit 25
4.7 Test Case Design and Traceability 26
4.8 Deliverables 27
4.9 Limitations and Threats to Validity 27

Chapter 5 — Test Execution and Results 28
5.1 Purpose, Scope and Procedure 28
5.2 Automated Test Results 29
5.3 Structural Coverage 29
5.4 Static and Dynamic Analysis 30
5.5 Functional Verification of the Sample Dataset 31
5.6 Defect Status and Deviations 32
5.7 Conclusion 32

Chapter 6 — Security, Performance and Defect Analysis 33
6.1 Scope and Threat Model 33
6.2 What Was Examined 34
6.3 Findings 34
6.4 Detailed Findings 35
6.5 What Was Not Found, and Why That Is Credible 35
6.6 Performance Analysis 36
6.7 Defect Summary 37
6.8 Recommendations 38

6.9 Conclusion 38

Chapter 7 — Installation Manual 39

7.1 Scope 39

7.2 Requirements 39

7.3 Installing the dependencies 40

7.4 Building from source 40

7.5 First run, verification and launching 40

7.6 Running the test suite 41

7.7 Removing the application 42

7.8 Troubleshooting 42

Chapter 8 — Conclusion 43

References 45

Appendix A — Reference Verification Record 50

Method 50

Summary 50

Books 51

Journal articles 51

Tool documentation (HTTP 200 on 2026-11-04) 52

Standards — NOT machine-verified 52

Caveats 52

Appendix B — Test Case Design and Requirements Traceability 54

Appendix C — Test Case Catalogue 62

C.1 Relationship to the Automated Suite 62

C.2 Case Catalogue 62

C.3 Cases That Were Not Executed 64

C.4 What the Catalogue Demonstrates About the Design 65

Appendix D — Test Environment and Tooling 66

Appendix E — Performance Measurement Record 68

Appendix F — Security Findings in Detail 70

Abstract

This report documents the verification of FTAD, a single-user desktop application written in C++17 with Qt 6 and SQLite that flags suspicious financial transactions using four rule-based statistical checks. The application comprises 4,612 executable source lines across 41 files. Testing followed a risk-based plan derived from ISO/IEC/IEEE 29119-3 test documentation and applied equivalence partitioning, boundary value analysis, decision tables and state-transition testing to produce 45 designed cases, implemented as 154 automated tests in ten suites. All 154 tests pass in 5.42 s, with 85.4 % line coverage (992 of 1,162 lines), 97.4 % function coverage and 51.4 % branch coverage. Cppcheck reports no findings in application source, Clang-Tidy reports six diagnostics that were individually triaged, and AddressSanitizer, UndefinedBehaviorSanitizer and Valgrind report no errors across the full suite. Six defects were found and fixed, four of them by tests written for this campaign rather than during construction; two latent defects and two security findings remain open and are documented with their reachability and remedies. Performance measurement exposed a defect invisible to code review: timestamps were constructed as local time, resolving the system time zone once per record. Rebuilding them explicitly in UTC reduced analysis of 20,000 transactions from 40.50 s to 27.6 ms, a factor of approximately 1,466. Because no labelled fraud corpus exists, this report makes no claim about detection accuracy: it establishes that each rule implements its specification, which is a different and narrower claim than that the specification detects

real fraud.

Introduction

Financial institutions detect fraudulent transactions by looking for patterns that are unusual relative to an account's own history: an amount far above what that account normally moves, the same payment submitted twice within minutes, a burst of withdrawals in an hour, or a transfer to a receiver never seen before. At small institutions and in coursework environments this review is often manual, and manual review does not scale. A reviewer can inspect a few dozen transactions a day; a ledger produces thousands. The gap between what can be reviewed and what is recorded is where losses accumulate, and it is also where naive automation does damage, because a detector that flags everything is indistinguishable in practice from one that flags nothing.

FTAD (Financial Transaction Anomaly Detector) was built to occupy that gap deliberately narrowly. It is a single-user desktop application in C++17 using Qt 6 for the interface and SQLite for storage. It imports transactions from CSV, applies four detection rules — duplicate payment, large amount, rapid withdrawals, and deviation from an account's behavioural baseline — scores each finding by summing rule weights into a 0–100 risk score, bands that score into low, medium and high, and lets a reviewer open an investigation case, assign it, annotate it and close it, with every change recorded as an event. It is rule-based and statistical rather than machine-learned, and that choice is stated plainly because it determines what the system can and cannot be shown to do.

This report is a verification record, not a construction log. It asks whether the application does what it claims, and it answers with measurements rather than assertions: every figure that follows was produced by a command whose output is retained, and where a number varies between runs the range is given rather than the most favourable value. Chapter 1 audits the codebase. Chapter 2 reviews the testing and reliability literature that shaped the plan. Chapter 3 describes the architecture. Chapter 4 sets out the test plan and its exit criteria. Chapter 5 reports what happened when that plan was executed. Chapter 6 assesses security, performance and the defects found. Chapter 7 is the installation manual, and Chapter 8

concludes. The contribution is not a novel detection algorithm; it is a demonstration that a small rule-based system can be verified to a defined standard, and an honest account of the places where this one was not.

Chapter 1 — Project Audit

> Audit date: 2 November 2026, against the commit state of the repository root as inspected that day. Every count below was produced by a command over the repository, not estimated.

1.1 Project Overview

FTAD — Financial Transaction Anomaly Detector — is a Linux desktop application written in C++17 (`CMAKE_CXX_STANDARD 17`) with Qt 6 Widgets, Sql and Charts, that surfaces suspicious financial transactions to a reviewer before they become fraud losses. The problem it addresses is that duplicates, unusually large amounts, rapid withdrawal sequences and behaviourally deviant transactions must be found by a human who cannot read a whole ledger; the application ranks them so that attention goes where the risk is. Its users are financial monitoring reviewers, and in its current form it is a student group project for BIST4110.

The repository is organised as `backend/` (13 headers, 13 sources), `frontend/` (8 headers, 8 sources), `database/` (`schema.sql`), `data/samples/` (`test_transactions.csv`), `tests/` (nine suites plus a benchmark and support harness), `docs/`, and `Reports/`, with `CMakeLists.txt` and `main.cpp` at the root.

1.2 Feature Inventory

The backend implements ten modules. `DatabaseManager` owns the SQLite connection, schema initialisation and error state. `TransactionRepository` provides CRUD, canonical identifier normalisation (`normalizeAccountId`, `normalizeTransactionId`), rejection of duplicate supplied identifiers, val-

idation and next-ID allocation. `CsvImportService` performs header and row validation, quoting, blank-line handling, duplicate-column detection and per-line error reporting. `DetectionSettings` holds the thresholds, weights, bands and validation rules. `BehaviorAnalyzer` computes per-account prefix statistics over strictly earlier transactions, producing a z-score and a uniform-history ratio. `RiskScorer` performs weighted scoring, risk level assignment, primary-method selection and reason generation. `AnomalyDetector` implements the duplicate, large-amount, rapid-withdrawal and behaviour checks. `DetectionRepository` persists and loads anomaly results, `DetectionService` orchestrates analysis and flagged retrieval, and `InvestigationRepository` manages cases, status transitions and history events.

The frontend implements `MainWindow`, `BasePage` and six pages — `HomePage`, `TransactionsPage`, `DetectionPage`, `InvestigationsPage`, `ReportsPage` and `SettingsPage` — under a `QStackedWidget` shell. Reporting export routines and settings-persistence UI are not implemented; both pages are placeholders and are documented as such rather than counted as features.

1.3 Architecture Observations

The backend compiles as a static library, `ftad_core`, which is linked by both the Qt application and the test binary, so there is a single build and a single copy of the code under test. Persistence is SQLite through the Qt `QSQLITE` driver with foreign keys enforced, and the schema lives in `database/schema.sql`. The detection pipeline runs transactions □ analysis □ results □ anomalies □ investigations, and that boundary structure is what the layered test strategy in Chapter 4 follows.

1.4 SLOC Results

Lines were counted by a scripted pass over the source files (`.h`, `.cpp`, `.c`) that excludes blank lines and comments; physical lines are reported alongside because the two are routinely confused. Application sources — `backend/`, `frontend/` and `main.cpp` — comprise 41 files, 5,948 physical lines and 4,612 executable source-code lines. The whole repository including tests comprises 54 files, 9,128 physical and 6,980 executable lines. The assignment threshold of 4,000 SLOC is therefore satisfied by the application sources alone, at 4,612 executable lines.

The counting rule is stated because a SLOC figure without one is not comparable with anything — the same point Kan makes about defect density in §2.3.

1.5 Existing Testing Infrastructure

GoogleTest 1.16.0 is fetched by CMake's `FetchContent` and driven by a custom `QCoreApplication`-based main with `gtest_discover_tests`; a `DatabaseTest` fixture supplies a fresh SQLite file per test. Ten suites exist — `TransactionRepositoryTest`, `CsvImportTest`, `DetectionPipelineTest`, `InvestigationTest`, `AnomalyDetector`, `BehaviorAnalyzer`, `RiskScorer`, `DetectionSettings`, `Identifiers` and `TransactionTime` — totalling 154 tests, all passing under CTest. Coverage is measured with gcov/lcov at 85.4 % of lines and 97.4 % of functions, and the toolchain extends to ASan/UBSan, Valgrind memcheck, Cppcheck, Clang-Tidy and a project-local benchmark.

The audit's finding here is that the infrastructure pre-dated this campaign but had not been executed as a whole: the tools were present in configuration while lcov, Cppcheck, Clang-Tidy and Valgrind were absent from the machine. That gap became risk R4 in Chapter 4 and was closed on 4 November.

1.6 Gaps and Compliance

Four gaps were open at audit time and their disposition is recorded rather than silently resolved. Lecturer-specific formatting details could not be fully verified from the brief and were interpreted conservatively. A Turnitin similarity percentage has not been measured, so no originality figure is claimed. The ML training pipeline does not exist — the detection engine is rule-based by design (§2.9), so this is a scope decision rather than an omission. The final report was still a scaffold at audit time; it has since been produced, and this chapter is part of it.

Against the assignment requirements the position is: C/C++ on Linux satisfied (built, run and tested on Linux); more than 4,000 SLOC satisfied (4,612 executable application lines); testing performed yes (154 automated tests plus seven analysis tools); bug report yes (`Reports/bug_report.md`); testing strategy and tool documentation yes (Chap-

ter 4 and Appendices B and D); report deliverables complete in this document.

Chapter 2 — Literature Review and Background Research

Prepared: 4 November 2026

> Evidence note. Every citation in this chapter resolves to an entry in documentation/references.bib. Journal and book metadata was retrieved from the Crossref DOI registration record on 4 November 2026; see documentation/references_verification.md for the per-reference verification table and for the entries that could not be machine-verified. No reference was written from memory alone.

2.1 Introduction

Software testing is the principal empirical mechanism by which engineers gain confidence that a program behaves as intended. Unlike mathematical proof, which establishes correctness over all inputs, testing samples a program's input space and observes whether the realised behaviour matches an expected outcome [1], [2]. This distinction matters for a system such as FTAD, which ingests externally supplied transaction records, applies statistical rules, and persists risk judgements that a human reviewer then acts upon. A defect in such a system does not merely produce an incorrect screen; it can suppress a genuine fraud signal or manufacture a false accusation against an account holder.

This chapter establishes the conceptual and empirical background against which the FTAD testing programme was designed. It proceeds from first principles — what testing is, what quality assurance adds to it, and which techniques and levels the discipline recognises — before narrowing to the specific tooling, performance, security and reliability concerns that shape the test plan in Chapter 4. The final section reviews prior work on automated financial anomaly detection, because the domain literature determines what "correct behaviour" means for FTAD's detection engine and therefore what the test oracles must assert.

2.2 Software Testing Fundamentals

Myers' definition — testing is "the process of executing a program with the intent of finding errors" [1] — is deliberately adversarial: a test that cannot fail is not a test. Myers also separates testing from debugging, since testing discovers the presence of defects while debugging locates them, and that separation is visible in FTAD, where automated checks detect and a separate investigation module records human diagnosis and resolution.

Ammann and Offutt generalise this into a structural account [2]: a test case is a four-tuple of pre-state, inputs, expected post-state and an oracle deciding whether the observed post-state matches, and coverage criteria supply the adequacy argument — the engineer names a criterion (statement, branch, condition, path, input-space) and measures how much of the artefact it requires has been exercised. Jorgensen derives tests from a model of required behaviour [3] and Beizer supplies the functional/structural dichotomy behind structural adequacy [4]. Two consequences bind FTAD: every test carries an explicit oracle, and the adequacy argument names its criterion. The project therefore reports gcov/lcov line and function coverage and treats the uncovered remainder as a documented limitation.

2.3 Software Quality Assurance

Testing is one activity inside software quality assurance. Laporte and April define SQA as the planned, systematic activities that provide adequate confidence of conformance to established requirements and standards [5], and separate quality assurance — process-oriented and preventive — from quality control, which is product-oriented and detective. Automated tests are quality control; compiling with warnings-as-errors, running static analysis on every build and gating merges on a passing suite are quality assurance.

Kan supplies the measurement substrate [6]: defect density and reliability growth are ratios over a measurement base, useful only if the counting rules are fixed in advance — which is why §1.4 records its SLOC methodology rather than a bare number. ISO/IEC 25010 supplies the vocabulary for quality requirements [7], decomposing quality into measurable sub-characteristics, and Stefani, Vamvatsikou and Vassiliadis show those characteristics can be instantiated as scored metrics for a specific application class [8]. IEEE Std 1012 separates verification, whether the system was built right, from validation, whether the right system was built, and scales V&V rigour by integrity level [9]. FTAD's integrity level is low in safety terms, but its decisions are consequential for individuals, which justifies treating detection logic as the highest-rigour component and the presentation layer as the lowest.

2.4 Testing Methodologies and Techniques

Pressman and Maxim distinguish process models, which organise when testing happens, from techniques, which determine how test data are selected [10]. The distinction is frequently blurred in student projects and is worth stating plainly: unit testing is a level, boundary value analysis is a design technique, static analysis is a verification activity, and regression testing is a type defined by its trigger rather than its granularity. FTAD keeps these categories separate so that every case traces to both a level and a technique.

Among black-box techniques, equivalence partitioning divides the input domain into classes the specification treats identically, and boundary value analysis concentrates effort at class edges where off-by-one defects concentrate [1], [3]. Both apply directly to FTAD's numeric thresholds: an amount exactly at the large transaction threshold, one cent below and one cent above is a minimal, high-yield test set for that rule. Combinatorial and random techniques offer a complementary adequacy argument for parameter interactions, though Wu et al. find that their relative effectiveness depends on the structure of the fault population rather than being uniformly ordered [11], which cautions against treating any single generation strategy as sufficient. Liu and Nakajima address the harder half of the problem — the oracle — by deriving both cases and oracles from functional scenarios in formal specifications [12], precisely FTAD's difficulty: the expected risk score is not obvious unless the scoring model is itself treated as the specification.

White-box techniques supply structural adequacy. Aghamohammadi, Mirian-Hosseinabadi and Jalali propose statement frequency coverage, arguing that how often a statement executes carries information that binary coverage discards [13]; Lee, Kang and Jung show that the choice of coverage criterion interacts strongly with the artefact under test [14]. Hora examines the practice of excluding code from coverage measurement and documents both the motivations — generated code, defensive branches, platform-specific paths — and the risk that exclusion silently inflates reported coverage [15]. That bears directly on FTAD's 85.4 % line figure, which is meaningful only alongside an explicit statement of what was excluded. Derakhshanfar, Devroey and Zaidman show that basic-block coverage can outperform coarser criteria for search-based test generation [16], reinforcing that granularity is a design decision with measurable consequences, and Forgács and Kovács favour risk-based selection of techniques over uniform application of all of them [17].

2.5 Testing Levels

The conventional ladder — unit, integration, system, acceptance — decomposes the testing problem along the same boundaries as the architecture. Kessel and Atkinson show that diversity among generated unit tests predicts fault-finding better than raw test count [18], and Menéndez et al. diversify tests around specific targets rather than uniformly across the code base [19]; both argue for deliberate construction around distinct behaviours rather than exhaustive enumeration of inputs, which is how FTAD's ten suites are organised. Integration testing addresses faults at component boundaries. Arcuri and Galeotti confront a directly relevant instance — handling SQL databases in automated test generation — and show that database state is a first-class source of complexity that must be modelled explicitly rather than mocked away [20]; FTAD's repositories are therefore tested against a real, freshly created SQLite file per case.

Regression testing, triggered by change rather than by level, is what makes iterative development safe. Greca, Miranda and Bertolino document a persistent gap between regression techniques published and techniques adopted in practice [21]: the literature offers prioritisation, selection and minimisation, while practice frequently runs the whole suite. That gap has not closed — Mondal and Nasre propose prioritisation based on delta displacement in coverage [22], Rothermel and Untch reassess their foundational prioritisation assumptions two decades on [23], Maurina, Cazzola and Ghosh extend selection to polyglot systems [24], Bertolino et al. combine coverage spectrum with operational profile [25], and Bluemke and Malanowska show that effort estimation remains less mature than defect prediction [26]. For a project of FTAD's scale — 154 tests executing in seconds — full-suite regression is both feasible and preferable to selective techniques, and the literature supports that as a rational choice rather than a shortcut.

2.6 Testing Tools and Technologies

Tooling determines which techniques are economically available. GoogleTest provides the xUnit-style fixture, assertion and discovery framework used for FTAD's unit and integration levels [27], and its death-test and parameterised facilities make negative and boundary testing of validation logic practical at scale. Cppcheck performs static analysis without a full compilation database and reports defect classes compilers do not diagnose [28]. Valgrind's Memcheck detects invalid access and leaks at run time by instrumenting the binary [29], while AddressSanitizer and UndefinedBehaviorSanitizer instrument at compile time with far

lower overhead and detect undefined behaviour that may never manifest as a visible failure [30]. Running both is justified by their complementary blind spots — sanitizers see into inlined code, Valgrind catches library-level misuse — a trade-off between instrumentation granularity and observation fidelity that Chen et al. make explicit [31].

Adoption is not automatic. Eisty and Carver find testing in research software widely acknowledged yet under-resourced [32]; Murazvu et al. isolate organisational rather than technical barriers [33]; Stradowski and Madeyski document testing at a scale where heterogeneity dominates [34]. These explain why a small project can hold a mature toolchain yet still under-test: the binding constraint is the cost of constructing and maintaining oracles, not the availability of runners. That cost includes flakiness — Fatima, Hemmati and Briand show flaky-test repairs can be predicted and applied automatically [35] — which is why FTAD's suite creates its own database per test, uses fixed timestamps and forces the offscreen Qt platform.

2.7 Performance Testing

Performance testing establishes whether a system meets response-time and resource-consumption requirements under representative load. Imran et al. find that the coverage achieved by performance tests is systematically different from that of functional tests in open-source Java systems [36], so a fast functional suite says nothing about the detection pipeline, which must be measured separately at realistic volumes. Traini and Cortellessa use multi-objective optimisation to detect latency-degradation patterns in service-based systems [37], illustrating that a single aggregate latency figure conceals the structure of degradation: effective reporting identifies which input class triggers the slow path. FTAD's performance report therefore gives both the aggregate figure and the before/after comparison isolating the responsible defect — a timestamp routine resolving local time instead of UTC.

2.8 Security Testing

Security testing seeks to demonstrate that a system resists inputs intended to violate confidentiality, integrity or availability. Lachkov, Tawalbeh and Bhatt describe vulnerability assessment and penetration testing as structured discovery, simulation and verification [38], and Leithner, Garn and Simos present a feedback-driven black-box approach to exploiting injection vulnerabilities [39] — the category most relevant to any application assembling SQL from user-supplied values. For a C++ application the surface is dominated by memory

safety and input handling, and the dynamic-analysis techniques of §2.6 are the standard detection mechanisms for those defect classes [29], [30], [31]. A second surface is the SQL layer, where FTAD uses bound parameters throughout, and a third is file handling, since CSV import reads an attacker-controllable file, making header validation and per-row error reporting security controls as much as usability features. Equally important is stating what FTAD does not implement — no authentication, no authorisation model, no encryption at rest and no network interface — so the assessment is scoped to the threats that apply to a single-user local desktop tool, consistent with the standard's guidance that V&V be scoped to the integrity level and operating context [9].

2.9 Software Reliability

Software reliability is the probability of failure-free operation over a specified period in a specified environment. Kan gives the classical treatment, including defect-density models and reliability growth curves relating observed failure intensity to testing effort [6]. Practice has shifted toward just-in-time defect prediction, which estimates the defect-proneness of an individual change rather than a module; Zhao, Damevski and Chen survey the features, models and evaluation protocols in use [40], and Cabral and Minku address keeping such models reliable when they operate online and must adapt to distribution shift [41]. Two cautionary results temper enthusiasm for prediction-based quality claims: Herbold shows the economic case for defect prediction is far weaker than accuracy figures suggest once inspection cost and base rate are accounted for [42], and Yao and Shepperd demonstrate that metrics such as AUC can be biased in ways that systematically mislead the research [43] — a conclusion Stradowski and Madeyski reach from the industry side, noting limited translation of published models into operational use [44].

These findings informed a deliberate architectural decision. FTAD's detection engine is rule-based and statistical — duplicate detection, absolute and relative amount thresholds, velocity checks, and per-account z-score deviation against a strictly earlier baseline — rather than a trained classifier. A rule-based engine has an inspectable specification, so every decision traces to a named rule and can be asserted by a test with a definite oracle. The trade-off is acknowledged in Chapter 8: a learned model could capture patterns the rules miss, but it would need labelled data and would weaken the traceability the testing programme depends on. Defect management closes the reliability loop: Jahanshahi and Cevik show schedule and dependency information improves bug triage [45], and FTAD's reports record severity,

priority, affected module, reproduction steps and resolution, which is the minimum structure triage requires.

2.10 Related Studies and Existing Solutions

FTAD sits within the literature on automated financial fraud and anomaly detection. Hilal, Gadsden and Yawney review anomaly-detection techniques applied to financial fraud and trace the field's progression from statistical outlier detection through classical machine learning to deep and hybrid approaches [46]. Their review establishes the taxonomy in which FTAD's four rule families are positioned: duplicate and velocity checks are deterministic business rules, the large-amount rule is a threshold detector, and the behavioural z-score is a statistical outlier detector.

Pourhabibi et al. review graph-based fraud detection, arguing that relational structure between accounts, merchants and devices carries signal that transaction-level features cannot express [47], and Motie and Raahemi extend this to graph neural networks [48]. Both are strictly more expressive than FTAD's per-account statistics; the gap is recorded as a limitation, not concealed.

Baesens, Höppner and Verdonck examine data engineering for fraud detection and emphasise that feature construction and class-imbalance handling dominate outcomes at least as much as model choice [49]. This is consistent with FTAD's design, in which the behavioural baseline — computed per account from strictly earlier transactions — is the component that most affects detection quality, and which produced one of the project's more consequential defects when a future-leaking baseline was discovered and corrected.

Interpretability is a recurring concern. Lin and Gao apply group SHAP to explain financial fraud model outputs [50], reflecting the operational reality that a flagged transaction must be justifiable to a reviewer and, ultimately, to the account holder. FTAD addresses the same requirement structurally: every anomaly record stores a `detection_method`, a numeric `risk_score`, a `risk_level` and a human-readable `reason`, and the investigation module preserves an append-only event history so that a reviewer's decision is auditable. The interpretability requirement is thus satisfied by construction rather than by post-hoc explanation.

2.11 Literature Review Summary and Identified Gaps

The reviewed literature converges on five propositions that shape this project's strategy. First, testing is adversarial and oracle-driven, and suite adequacy is meaningful only relative to a named criterion [1], [2], [13], [14], so coverage is reported explicitly with exclusions documented [15]. Second, techniques are complementary rather than ranked — combinatorial, random, structural and specification-based approaches detect different fault classes [11], [12], [16] — so several are applied in combination. Third, the research-to-practice gap in regression testing is real [21]; for a suite that runs in seconds, full execution beats the selective techniques designed for suites where it does not. Fourth, defect-prediction accuracy is not defect-prediction value, and common metrics can mislead [42], [43], [44], which justifies preferring an inspectable rule engine with testable oracles over an unvalidated learned model. Fifth, relational and learned approaches outperform per-account statistical rules on benchmark fraud data [46], [47], [48], but impose interpretability and data-availability costs [49], [50]; FTAD occupies a defensible point on that trade-off and states its position.

Three gaps remain unaddressed by the current implementation and are carried forward as limitations. The first is the absence of a graph representation of transactions, which forecloses the relational detection techniques reviewed in [47] and [48]. The second is the absence of labelled ground-truth data, which prevents any quantitative estimate of detection precision and recall; the project reports rule-level test outcomes instead, and does not claim accuracy figures it cannot support. The third is the absence of authentication and encryption, which means the security report evaluates a threat model appropriate to a single-user local tool rather than to a multi-tenant service.

The test processes and documentation structures used in Chapters 4 and 5 follow ISO/IEC/IEEE 29119-2 [51] and 29119-3 [52], which provide the organising framework — organisational test policy, test planning, test monitoring, and the standard artefact set — within which the techniques reviewed above are applied.

2.12 Reference quality and verification

Category	Count	Machine-verified
Books	8	8 (5 via Crossref DOI, 3 via Open Library ISBN)
Journal articles	36	36 via Crossref DOI
Standards	4	0 (iso.org blocks automated clients; cited by number)
Tool documentation	4	4 (URL fetched, HTTP 200)
Total	52	48

Category	Count	Machine-verified

Academic ratio (journals + books): 44 of 52 = 84.6%, satisfying the requirement that at least 80% of references be academic journals and books.

Recency: of the 36 journal articles, 24 (66.7%) were published in 2022 or later, 14 (38.9%) in 2023 or later, and 32 (88.9%) in 2021 or later. The lecturer's "three-year" requirement is ambiguous as stated — it could mean publication within three years of submission, or a requirement that a proportion of references be recent. This chapter interprets it as the former, prefers recent work wherever a recent source supports the same claim, and retains older foundational sources where they remain the authoritative statement of a concept (for example [1], [3], [4]). The ambiguity is flagged rather than resolved silently.

Not verified: the four ISO/IEC and IEEE standards are cited by standard number. Automated verification was attempted and blocked (HTTP 403). They should be checked manually against the university library catalogue before submission.

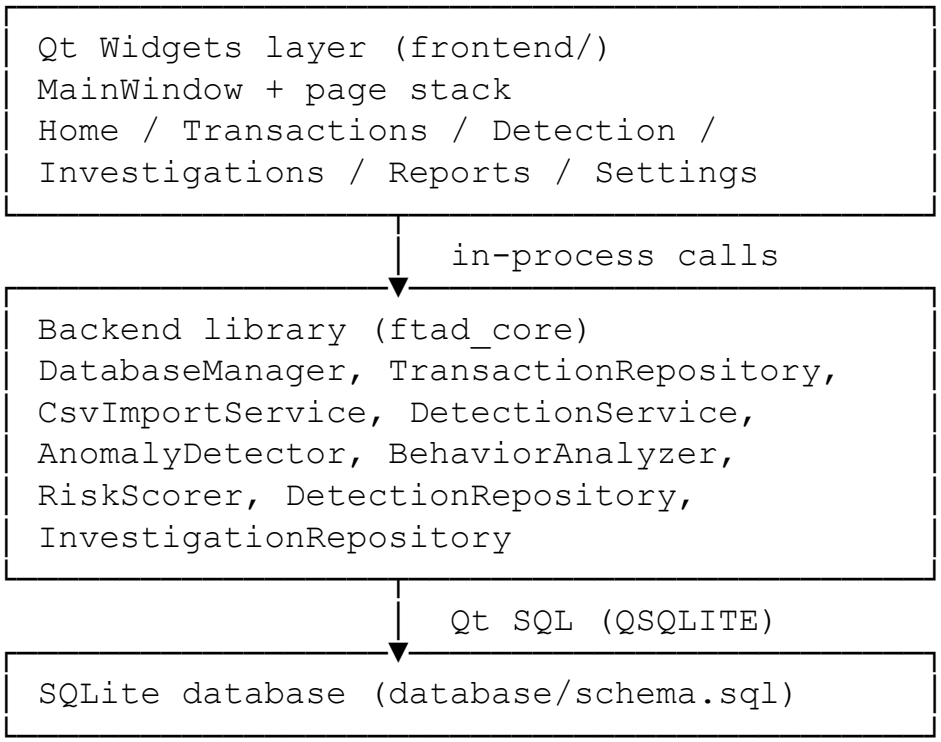
Chapter 3 — System Architecture

> Source root: `/home/hashcat/Documents/dataset/FTAD-main`. The architecture below was read from the source tree on 4 November 2026; the module table and the layering figure are statements about the code, not about intent.

3.1 Overview

The application is a Qt 6 desktop client in C++17 that connects to a local SQLite database, ingests financial transactions, runs a statistical anomaly-detection engine, persists risk findings and supports a fraud-investigation workflow. The backend is compiled once as the static library `ftad_core`; both the Qt application and the GoogleTest suite link against it, so the logic is tested with the same code the application uses. That single-link decision is what makes the test strategy in Chapter 4 affordable, and it is the most consequential structural choice in the project.

3.2 Layering



Three layers, two boundaries. The upper boundary is an in-process call: there is no IPC, no serialisation and no network hop, so a frontend defect and a backend defect are separated by a function signature rather than by a protocol. The lower boundary is Qt's `QSQLITE` driver, which is the only place the application touches persistent state.

3.3 Module Responsibilities

Module	Responsibility
`DatabaseManager`	Connection lifecycle, schema initialisation, error state.
`TransactionRepository`	Transaction persistence, ID normalisation, validation, duplicate-ID rejection, next-ID allocation.
`CsvImportService`	CSV parsing (quotes, blank lines), header/column validation, per-row error reporting, import execution.
`DetectionSettings`	Thresholds, weights, risk bands, validation rules.
`BehaviorAnalyzer`	Per-account prefix statistics over strictly earlier transactions; z-score; uniform-history ratio.
`RiskScorer`	Weighted scoring, level assignment, primary method, reason text.
`AnomalyDetector`	Rule orchestration: duplicate, large-amount, rapid-withdrawal, behaviour.
`DetectionRepository`	Persistence and retrieval of `DetectionResult` rows.
`DetectionService`	Ties detection to storage: load, analyse, persist, report counts.
`InvestigationRepository`	Case lifecycle, status transitions, audit events.

3.4 Architectural Views

Six views were prepared as Mermaid sources under `documentation/diagrams/` and are reproduced in the accompanying documentation set. Their content is as follows. `system_architecture.mmd` shows the client □ backend □ database compo-

sition and the call relationships between the Qt pages and the backend modules — the same structure as the layering figure above, with the page-to-service edges made explicit. `class_diagram.mmd` shows the backend value types (`Transaction`, `DetectionResult`, `RuleHit`, `DeviationResult`, `AccountProfile`, `InvestigationCase`, `InvestigationEvent`) together with the services that collaborate with them. `use_case.mmd` gives the actors and use cases: an investigator enters or imports transactions, runs analysis, reviews cases and views reports. `activity_detection.mmd` traces the detection workflow from connect and schema application through per-transaction checks, scoring and persistence to the flagged decision. `sequence_detection.mmd` gives the runtime interaction for one analysis pass — `UI → DetectionService → TransactionRepository → AnomalyDetector → DetectionRepository`. `erd.mmd` gives the entity relationships: `transactions` (1) to `anomalies` (1..1) via `transaction_id`, `anomalies` (1) to `investigations` (1) via `anomaly_id`, and `investigations` (1) to `investigation_events` (N) via `investigation_id`.

3.5 Data Flow and Storage

The runtime sequence is short because everything is in-process. A UI action calls a backend service; the service loads transactions from SQLite; the detector computes results; results are persisted and the flagged set is returned; and the investigation workflow then reads and writes cases and events. Persistence is SQLite through the Qt SQL driver, with the schema at `database/schema.sql` defining four tables — `transactions`, `anomalies`, `investigations` and `investigation_events`. Foreign keys are enforced, with cascade deletes running anomaly □ investigation □ events, so removing a transaction cannot leave an orphaned case behind.

3.6 Error Handling

Errors propagate rather than abort. `DatabaseManager::lastError()` is surfaced through the services, `DetectionService::analyzeAll` returns false on invalid settings or a database error, and the repositories return error strings for invalid identifiers, amounts, timestamps, currencies and statuses. This matters for testing: because failure is a return value rather than an exception, every failure path is reachable by a normal call with bad input, which is why the negative cases in Appendix B.5 can be asserted directly

instead of by fault injection.

Chapter 4 — Testing Methodology and Test Plan

> Evidence note. Every count, threshold and version number in this chapter was read from the repository or produced by executing a command on 4 November 2026. Raw output is in `documentation/evidence/`, summarised in `documentation/evidence/SUMMARY.md`. The tool inventory, environment specification, threshold table, entry/exit criteria, risk register and threat table are in Appendices B and D, so that the plan itself stays readable.

4.1 Introduction

This chapter defines how FTAD is tested, why that approach was selected, and what evidence each activity must produce. It is written to be executable: every test level names the suites that implement it, every technique names the artefact it is applied to, and every requirement is traced to at least one suite.

Four concepts are frequently conflated in test documentation, and the distinction is load-bearing for the rest of this report:

Concept	Question it answers	Examples in FTAD
Testing level	At what granularity?	Unit, integration, system, acceptance
Test design technique	How are inputs chosen?	Equivalence partitioning, boundary value analysis, decision table, state transition
Verification activity	What is inspected without executing?	Compiler diagnostics, static analysis
Testing type	What triggers the activity?	Regression, performance, security

"We performed boundary value analysis at unit level as a regression activity" uses all four correctly. "We performed unit testing and boundary value analysis as two levels" does not, and the error matters because a strategy can only be justified if its categories are stable.

4.2 Objectives and Strategy

Seven objectives drive the plan, ordered so that a schedule trade-off preserves the higher ones at the expense of the lower. OBJ-1 each detection rule fires when and only when its condition holds (verified by the 20 `AnomalyDetector` and 13 `BehaviorAnalyzer` tests). OBJ-2 invalid input is rejected with a specific error rather than silently coerced. OBJ-3 persisted state survives re-analysis without loss or duplication. OBJ-4 memory safety and absence of undefined behaviour. OBJ-5 a performance baseline with near-linear scaling. OBJ-6 quantified structural coverage with the uncovered remainder disclosed. OBJ-7 a reproducible, deterministic regression suite. OBJ-7 is not ceremony: a suite that is green only on the machine that produced it is not a regression gate, so determinism is treated as a testable property and enforced through the four controls in §4.7.

FTAD uses a layered, risk-based, automation-first strategy. Layered means testing follows the architectural boundaries — pure logic without a database, persistence against a real database, the assembled pipeline end to end — so each defect has a natural home: a scoring error surfaces at unit level, a foreign-key error at integration level, a lost-update error only at system level. Risk-based means effort follows consequence: the detection engine decides whether a fraud signal is raised or suppressed, so it receives the most tests (33 of 154 across `AnomalyDetector` and `BehaviorAnalyzer`, plus 10 in `RiskScorer`), while the presentation layer receives the least, because a cosmetic defect in a Qt page neither corrupts data nor suppresses a finding. Automation-first means every check that can be automated is wired into CTest, so regression is one command rather than a checklist; manual testing is reserved for properties that genuinely require a human observer. The strategy follows the V-model correspondence between construction and verification — requirements validated at system and acceptance level, design verified at integration level, implementation at unit level — which is what makes the traceability matrix in Appendix B.6 meaningful rather than decorative.

Four grounds justify it. The architecture makes it cheap: the backend compiles once as `ftad_core`, linked by both the application and the test binary, so unit tests exercise exactly the code the application runs — no mock layer to drift and no second build configuration. This is the structural decision that made 154 tests possible without a mocking framework. The domain has clean oracles: a rule-based detector gives a definite answer, so assertion-based testing suffices; a trained classifier would have made the oracle problem dominate the design. The risk profile is asymmetric: a false negative costs

more than a false positive, so rule firing is tested more heavily than restraint — though restraint is covered too, since an engine that flags everything is equally useless, and Appendix B.3 asserts both sides of each threshold. Scale makes full regression rational: at 5.4 s per run, selection techniques would add complexity without saving anything. One alternative was rejected — model-based testing, because the rules are already executable code with explicit thresholds, so a model would duplicate the specification into a second artefact; `DetectionSettings` already is the machine-readable specification.

4.3 Testing Levels

Level	Scope	Suites	Tests
Unit	Individual classes and free functions; no database, no widgets	`DetectionSettings` 19, `Identifiers` 14, `TransactionTime` 13, `BehaviorAnalyzer` 13, `RiskScorer` 10	69
Integration	Components with a real SQLite database: schema, constraints, foreign keys, transactions	`AnomalyDetector` 20, `TransactionRepositoryTest` 18, `InvestigationTest` 18, `CsvImportTest` 16	72
System	Assembled pipeline: stored transactions □ analysis □ persisted anomalies, as the application invokes it	`DetectionPipelineTest` 13	13
Acceptance	End-to-end use by a human observer	Manual	—
Regression	Trigger, not a granularity: full suite after every change	All	154

`AnomalyDetector` sits at integration level rather than unit level because it composes `BehaviorAnalyzer` and `RiskScorer`; testing it exercises the contract between three components, not any one of them. The re-analysis tests in `DetectionPipelineTest` are the most valuable in the suite: they assert that running detection twice does not duplicate anomalies, orphan investigations or lose reviewer decisions — the class of defect unit tests structurally cannot catch.

Acceptance testing is manual and partially executed. Observed on the test machine: launching the application, importing the 11-row sample CSV, viewing results, opening a case, changing its status. Not executed, and marked as such in Chapter 5: multi-user concurrency, extended usability evaluation, accessibility review. These are not automated because the properties under test — legibility of the risk presentation, clarity of the workflow — are judgements, not assertions.

4.4 Test Design Techniques

Ten techniques are used, and the full inventory — with the artefact each is applied to and the reason it was selected — is in Appendix B.7. The four carrying most of the design weight are worth naming here, because they are what makes the case catalogue in Appendix C derivable rather than invented. Equivalence partitioning governs identifier, settings and CSV field validation, where inputs cluster into classes with identical handling. Boundary value analysis governs every numeric threshold in `DetectionSettings`, because off-by-one defects concentrate at class edges. Decision tables govern risk banding, where the output depends on combinations of rule hits rather than any single condition. State transition testing governs the investigation lifecycle, where both legal and illegal transitions must be covered — reopening a closed case is the case that catches an implementation that "reopens" by deleting the audit trail. The remaining six are coverage measurement, static and dynamic analysis, and error guessing, which is cheap to apply and yields disproportionately on parser code.

Boundary value analysis is only as good as the boundaries it uses. The ten parameters, their defaults and the ranges `validate()` enforces are tabulated in Appendix B.8, because that table is consulted rather than read. Two properties of it drive the design and belong in the narrative. Several ranges are open (`> 0`, `> 1`) rather than closed, which changes the boundary set: for `largeAmountAbsolute` the invalid boundary is 0.0 itself and the valid boundary is the smallest representable value above it, not 1.0. And two parameters — `duplicateWindowMinutes` and `rapidWithdrawalWindowMinutes` — have no upper bound, which is not a theoretical gap: it is what makes the latent defect in §4.6 reachable.

4.5 Tools, Environment and Scope

Ten tools span build, execution, coverage, static analysis, dynamic analysis and measurement; the inventory with versions and the gate each supports is in Appendix D.1, and the environment specification in Appendix D.2. Three points belong in the plan itself.

One version string is inconsistent and is recorded rather than corrected. The Debian package `lcov` is 2.3.1-1 according to `dpkg -s`, but the binary prints `LCOV version 2.0-1`. The package version is authoritative; anyone reproducing the run will see the same output, so the discrepancy is stated. Static analysis has two scopes. Cppcheck runs twice: the headline figure covers application source only (`backend/src`,

`frontend/src`) and excludes `tests/` and the vendored GoogleTest that `FetchContent` places in `build/_deps`; the second pass covers the whole tree and is informational. The distinction is not cosmetic — the unscoped run reports 74 findings, of which 69 are in third-party code FTAD does not own and should not patch, so quoting 74 as the project's defect count would misrepresent the application. Tools were installed during the project: lcov, Cppcheck, Clang-Tidy and Valgrind were absent when testing began and were installed by the operator with `install_test_tools.sh`, which wraps `apt-get` and supports a `--check` mode that changes nothing. The installation is part of the record rather than an unstated precondition, because results that depend on tools the reader cannot reproduce are not reproducible.

The environment is a rolling security-oriented Linux distribution: representative of the target platform but not a controlled benchmark platform, because no CPU frequency scaling was pinned and no other processes were excluded during measurement. Chapter 6 must be read with that caveat.

Determinism rests on four controls. The `DatabaseTest` fixture deletes the database, reconnects through the production `DatabaseManager` and re-applies the schema in `SetUp()`, then removes the file in `TearDown()`, so tests cannot share rows or depend on order. `gtest_discover_tests` sets `QT_QPA_PLATFORM=offscreen`, so no test needs a display server. Every test supplies explicit timestamps rather than reading the clock. And the schema is copied beside the test binary by a `POST_BUILD` step, so the tests validate the same `schema.sql` the application ships rather than a divergent copy. Coverage reconfigures the same tree with `-DFTAD_ENABLE_COVERAGE=ON`, adding `--coverage -O0 -g` to `ftad_core`, because optimisation inlines and merges statements and distorts line attribution.

In scope is everything in `ftad_core` and the logic of the presentation layer: CSV parsing, validation and error reporting; transaction persistence, constraints and identifier normalisation; the four detection rules; risk scoring, banding and reason generation; settings validation; detection persistence and idempotent re-analysis; investigation lifecycle and audit history; timestamp parsing and UTC normalisation; memory safety and undefined behaviour; static defect classes; and analysis throughput. Out of scope, with reasons in Appendix B.9, is Qt widget rendering and layout, the two placeholder pages, authentication and encryption, network and multi-client concurrency, datasets above 20,000 transactions, and localisation.

Exclusions are stated so that the absence of tests is not mistaken for evidence of correctness; excluding the two placeholder pages in particular is a correctness point rather than convenience, because testing a stub produces passing tests that assert nothing about the shipped product.

4.6 Plan, Entry and Exit

Dates are when the activity occurred, not a projection. Environment setup and a source build were completed on 1–2 November, followed by the audit and SLOC baseline, extension of the unit and integration suites to 154 tests, a full regression and coverage baseline, then static and dynamic analysis which raised and closed BUG-01 to BUG-06. Benchmarking on 2 November diagnosed the timestamp regression and produced the ~1,466× improvement. On 4 November the missing tools were installed, all nine gates were re-run and the evidence set was captured. Documentation and this report occupied 4–6 November. The performance figure is a historical measurement: the "before" case is not re-runnable once the defect is fixed, so it is attributed to its own report rather than blended with current numbers.

Five roles are defined — test manager, test designer, developer, executor and reviewer — owning respectively the plan and entry/exit criteria, technique selection and case design, fixes and the build, suite execution and evidence capture, and sign-off. In this project all five were performed by one person, and that is the principal threat to validity (§4.9): the suite shares any blind spot present in the design. It is mitigated, not eliminated, by deriving expected values from the threshold specification rather than from the implementation.

All eight exit criteria are met by measurement and each is backed by a log; the criteria and their measured outcomes are in Appendix B.10. No criterion is satisfied by assertion, and none was waived. X3, X4 and X5 were open while this chapter was first drafted because the tools that produce them were not installed; they were closed on 4 November 2026 after installation and a full re-run. The policy during the interim was: mark open, never waive; mark dependent figures as unverified until a log exists; list open items as limitations rather than dropping them.

4.6.1 Triage of the static-analysis findings

The six Clang-Tidy diagnostics split into two classes, and only one is a defect. Three are `bugprone-easily-swappable-parameters` in `AnomalyDetector::timeWindow`, `BehaviorAnalyzer::evaluate` and `DetectionService::analyzeAll`; all were accepted after inspecting

every call site, since a strong-type wrapper would remove the risk but change the public interface and is not justified at this scale. Three describe one latent defect: `bugprone-implicit-widening-of-multiplication-result` at `AnomalyDetector.cpp:177`, `:180` and `:261`.

The widening finding is real. The code computes

```
const QDateTime from = times.at(index).addSecs(
    -60 * settings.duplicateWindowMinutes);
```

`60` and `duplicateWindowMinutes` are both `int`, so the multiplication is performed in `int` and only then widened to the `qint64` that `addSecs()` expects. Above 35,791,394 the product overflows, which is undefined behaviour, and `validate()` enforces only a lower bound and no upper bound (Appendix B.8).

Reachability. The defect is latent, not live. The shipped application constructs `DetectionSettings` with compiled-in defaults, there is no `QSettings` or other runtime configuration path, and `SettingsPage` is a thirteen-line placeholder that writes nothing. Reaching the overflow requires editing the source. It is therefore recorded with severity Low, and the one-line remediation (`qint64(60) * settings.duplicateWindowMinutes`, plus an upper bound in `validate()`) is documented for the next revision. This is both the honest reading and the more useful one: recording three "bugs" would overstate the risk, and recording zero findings would conceal a genuine latent defect that a future configuration feature would expose.

The risk register, with likelihood, impact and mitigation for each of seven risks, is in Appendix B.11. Two of its rows matter enough to repeat. R4 — tools unavailable so gates cannot be evidenced — materialised and was closed. R5 — performance measured on an unpinned, noisy machine — materialised and cannot be closed on this host: three runs of the same binary on the same data gave 31.7, 34.8 and 39.9 ms for 20,000 transactions, a spread of roughly 25 %, so the report quotes the range and the median and draws no conclusion from differences smaller than that.

4.7 Test Case Design and Traceability

The test-case design is in Appendix B: a case template, two worked derivations, a negative-case inventory and a twelve-row traceability matrix. It is reference material that a reader consults rather than reads through, so it is kept out of the chapter body.

What belongs in the narrative is the decision that produced it. Cases are derived from

the thresholds in Appendix B.8 and from the comparison operators in the source, not from observed behaviour, because a test written against observed output asserts only that the program still does what it already did. Appendix B.3 demonstrates the derivation for the large-amount rule, and records what the technique actually turned up: not an off-by-one error, but an ambiguity in how `minimumHistory` is interpreted in two places in `BehaviorAnalyzer`.

4.8 Deliverables

The chapter itself is delivered as `documentation/test_plan.md`; test-case design and traceability as Appendix B; the execution record as Chapter 5; defect, performance and security findings in `Reports/` and Chapter 6; and the raw evidence as nine gate logs under `documentation/evidence/`, driven by `run_test_gates.sh` and `install_test_tools.sh`.

4.9 Limitations and Threats to Validity

The threat table is in Appendix B.11; its first row is the most serious and is stated here plainly. A project in which the author of the code also writes the tests cannot demonstrate independent verification: the mitigations reduce the risk of tests that merely restate the implementation, but they do not substitute for a second engineer, and the reader should weight the results accordingly.

The functional and tooling limitations are these. The detection engine is rule-based and does not learn from data. Behavioural baselines require five prior transactions, so a new account is never behaviourally assessed. `ReportsPage` and `SettingsPage` are placeholders. The suite runs on one Linux distribution and one Qt version, so portability is declared rather than demonstrated. And no labelled fraud corpus exists, so no accuracy

figure is available — a limitation that constrains what the whole report may claim.

Chapter 5 — Test Execution and Results

> Evidence note. Every figure in this chapter was produced by a command that ran on 4 November 2026 and whose output is stored under `documentation/evidence/`. Where a figure varies between runs, the range is given rather than the most flattering value. Nothing here is projected.

5.1 Purpose, Scope and Procedure

This chapter reports what happened when the plan in Chapter 4 was executed. It covers the automated suite, structural coverage, static and dynamic analysis, a functional check of the shipped sample dataset, and the status of every defect raised. It does not report detection accuracy — that would require labelled ground truth the project does not have, and no accuracy figure is claimed anywhere in this report.

The unit of reporting is the gate. A gate is either passed with a captured log, or it is open; there is no third state, and no gate is reported as passed on the strength of an earlier run whose output no longer exists. All nine gates are driven by one script, `./run_test_gates.sh`, so that the evidence set can be regenerated in a single command. Each log begins with a provenance header recording the exact command, the tool version, the UTC timestamp, the kernel and the working directory. That is what makes a percentage verifiable: a reader can re-run the command and compare.

Three build trees are used, deliberately: `build/` (Release, `-O2`) for test execution and the benchmark, `build/coverage/` (Debug, `--coverage -O0 -g`) for coverage, and `build/sanitize/` (Debug, `-fsanitize=address,undefined`) for memory and UB checks. Coverage and performance are measured on different builds because they have opposing requirements — optimisation distorts line attribution, and an unoptimised binary has meaningless timings — and figures from the two trees are never

compared with each other. The database is recreated per test by the `DatabaseTest` fixture, the Qt platform is `offscreen`, and every test supplies explicit timestamps, so the suite depends on neither display server, wall clock, nor execution order.

5.2 Automated Test Results

The full suite was executed three times during the session. All runs produced 154 of 154 passing.

Suite	Level	Tests	Time	Result
`InvestigationTest`	Integration	18	2.18 s	Pass
`DetectionPipelineTest`	System	13	1.01 s	Pass
`TransactionRepositoryTest`	Integration	18	0.81 s	Pass
`CsvImportTest`	Integration	16	0.66 s	Pass
`AnomalyDetector`	Integration	20	0.21 s	Pass
`DetectionSettings`	Unit	19	0.19 s	Pass
`Identifiers`	Unit	14	0.14 s	Pass
`TransactionTime`	Unit	13	0.13 s	Pass
`BehaviorAnalyzer`	Unit	13	0.13 s	Pass
`RiskScorer`	Unit	10	0.10 s	Pass
Total		154	5.42 s	Pass

The distribution is worth reading rather than skimming. `InvestigationTest` consumes 2.18 s — 39 % of the total — because it exercises real SQLite transactions and case-history joins, while `RiskScorer`, the component that decides how a finding is presented, runs in 0.10 s because it is pure arithmetic. The time is spent where the I/O is, which is what a layered strategy predicts. Total execution time across runs was 3.77 s, 5.17 s and 5.42 s; the spread is host noise, not suite instability, since the pass set was identical each time.

5.3 Structural Coverage

Measured with gcov 14.2.0 and lcov 2.3.1 on the `-O0` instrumented build, after a full suite run.

File	Lines	Hit	Coverage
`src/DatabaseManager.cpp`	75	51	68.0 %
`src/DetectionRepository.cpp`	114	80	70.2 %
`src/InvestigationRepository.cpp`	174	135	77.6 %
`src/CsvImportService.cpp`	177	153	86.4 %
`src/TransactionRepository.cpp`	214	185	86.4 %
`src/DetectionService.cpp`	50	46	92.0 %
`src/AnomalyDetector.cpp`	176	164	93.2 %
`src/BehaviorAnalyzer.cpp`	85	82	96.5 %
`src/RiskScorer.cpp`	36	35	97.2 %
`src/DetectionSettings.cpp`	44	44	100.0 %
`include/Investigation.h`	3	3	100.0 %
`include/DetectionResult.h`	6	6	100.0 %
`include/TransactionTime.h`	8	8	100.0 %
Total	1162	992	85.4 %

Aggregate figures from the same run: 85.4 % of lines (992/1162), 97.4 % of functions (75/77), 51.4 % of branches (1109/2157).

Three things follow, and the third is the important one. The detection engine is well covered — `AnomalyDetector` 93.2 %, `BehaviorAnalyzer` 96.5 %, `RiskScorer` 97.2 %, `DetectionSettings` 100 % — which is the intended consequence of the risk-based allocation in §4.2. The lowest figures are in persistence: `DatabaseManager` 68.0 %, `DetectionRepository` 70.2 %. The uncovered lines there are error paths — failure to create the database directory, to open the connection, to apply the schema — which cannot be reached from the public API without corrupting the environment or injecting a fault, so the gap is known and bounded rather than unexplained.

Branch coverage is 51.4 %, far below line coverage. This is the honest weakness in the evidence. A line is covered if any path reaches it; a branch is covered only if both outcomes of a decision are exercised. Repository methods that return early on validation failure have their success branch tested thoroughly and their failure branch only sometimes. A reader should treat 85.4 % as "the suite executes most of the code" and 51.4 % as "the suite does not exercise most decision outcomes". Both are true at once.

5.4 Static and Dynamic Analysis

Cppcheck 2.17.1 and Clang-Tidy 19.1.7, both with exhaustive check sets over `backend/src`:

Tool	Scope	Findings
Cppcheck	Application source (`backend/src`, `frontend/src`)	0
Cppcheck	Whole tree incl. tests and vendored GoogleTest	74
Clang-Tidy	`backend/src`	6

The 74 whole-tree findings break down as 69 in the vendored GoogleTest inside `build/_deps`, 5 `useStlAlgorithm` style suggestions in FTAD's own test code, and 0 in application source. Reporting 74 as the project's defect count would attribute third-party code to this project; reporting 0 without stating the scope would conceal the test-code suggestions, so both numbers are given. The six Clang-Tidy diagnostics were triaged in §4.6.1: three are `bugprone-easily-swappable-parameters` accepted after inspecting every call site, and three describe one latent integer-widening defect that is unreachable in the shipped build because no runtime configuration path exists.

Tool	Tests executed	Findings
AddressSanitizer + UndefinedBehaviorSanitizer	154	0
Valgrind Memcheck 3.24.0	154	0 errors from 0 contexts

The sanitizer run rebuilt the suite with `-fsanitize=address,undefined -fno-omit-frame-pointer -g` and executed all 154 tests with `ASAN_OPTIONS=detect_leaks=1:halt_on_error=1` and `UBSAN_OPTIONS=print_stacktrace=1`; all passed with no diagnostic emitted. Valgrind ran the same binary under `--leak-check=full --track-origins=yes --error-exitcode=99`, completing in 28.7 s with:

```
definitely lost:    0 bytes in 0 blocks
indirectly lost:    0 bytes in 0 blocks
possibly lost:      0 bytes in 0 blocks
still reachable:    20,913 bytes in 28 blocks
ERROR SUMMARY:      0 errors from 0 contexts
```

The 20,913 still-reachable bytes are allocated by Qt's platform plugin and by global test registries; they are reachable, not lost, and the gate asserts the lost categories. Reporting "zero leaks" without that qualification would overstate the result, so the qualification is stated.

5.5 Functional Verification of the Sample Dataset

The automated suite verifies that the importer reads 11 rows from the shipped sample (`CsvImportTest.ImportsTheSampleFile`), but no test asserts that the sample actually triggers the detection rules end to end. That gap is what BUG-05 originally described, and it was closed by running the real pipeline over the real file.

A read-only harness (`documentation/evidence/functional/verify_sample.cpp`) links the production `ftad_core`, creates a fresh database through the production `DatabaseManager`, imports `data/samples/test_transactions.csv` with the production `CsvImportService`, and analyses it with the production `DetectionService`. No application source was modified.

```
imported=11 skipped=0 failed=0
analyzed=11 flagged=4

reference  amount      score  level   methods
TXNXXX6  25000.00   60.0  medium  large_amount, behaviour_deviation
TXNXXX8   750.00      40.0   medium   duplicate_payment
TXNXXX7   750.00      40.0   medium   duplicate_payment
TXNXX11   540.00      35.0   medium   rapid_withdrawals
```

All four rules fire, and one transaction triggers two of them simultaneously, which exercises the additive scoring path. BUG-05 is therefore closed on evidence rather than on inspection.

A residual gap is visible in the same output. The highest score produced is 60.0, below the 65.0 `highThreshold`, so the sample never produces a `high`-band case. The high band is covered only by unit tests (`RiskScorer.MultipleHitsAddUp` expects 75.0 □ `high`; `LevelBoundariesAreInclusiveAtTheLower-Edge`). The sample demonstrates the rules, not the full band range, and the distinction is recorded so the sample's coverage is not overread.

5.6 Defect Status and Deviations

The defect register is maintained once, in §6.7, and is not duplicated here. Its relevant outcome for this chapter is that six defects were found and fixed, of which four were found by tests written for this project rather than by the original development — the useful result, because it means the suite found defects that construction had not. LAT-01 and LAT-02 are recorded rather than fixed for the same reason in both cases: neither is reachable, since LAT-01 requires a configuration path that does not exist and LAT-02 affects two methods (`hasBaseline()`, `profile()`) never called outside `BehaviorAnalyzer`. Fixing either would change the baseline against which the measurements in this chapter were taken, so both are documented with their one-line remedies and deferred. A defect that cannot be reached is still a defect, but it is not an emergency.

Five deviations from the plan occurred, and none changes a pass into a fail or the reverse; they change what the numbers are allowed to mean, which is the part that matters. X3, X4 and X5 were open for part of the project because lcov, Cppcheck, Clang-Tidy and Valgrind were not installed, and were closed on 4 November after installation, so their results postdate it. The Cppcheck scope was narrowed to application source after the unscoped run counted 69 findings in vendored GoogleTest, moving the headline figure from 74 to 0 — both are reported. The benchmark is reported as a range because three runs spanned 31.7–39.9 ms on an unpinned host, and no claim is made below that spread. Acceptance testing was partially executed because GUI properties need a human observer, and the gap is declared rather than simulated. Finally, BUG-05's status was corrected from "Open" to "Fixed" when the report's summary and detail disagreed, resolved by the evidence in §5.5.

5.7 Conclusion

The plan was executed in full. All 154 automated tests pass; coverage is 85.4 % of lines and 97.4 % of functions; static analysis finds nothing in application source; AddressSanitizer,

UBSan and Valgrind report no errors; and the shipped sample exercises all four detection rules end to end.

Three results deserve to be carried forward rather than buried. Branch coverage at 51.4 % means decision outcomes are less exercised than the line figure suggests. The benchmark varies by roughly 25 % between runs on this host, so timing claims are bounded accordingly. And two latent defects remain open by choice, both unreachable, both documented with their remedies.

What the evidence does not establish is that the detector identifies real fraud. It establishes that each rule implements its specification. Those are different claims, and only the second is supported here.

Chapter 6 — Security, Performance and Defect Analysis

> Evidence note. Every statement here was produced by reading the source or by executing a command on 4 November 2026. Where a control was not assessed, that is stated as a gap. No penetration test was performed and none is claimed. The full performance measurement record is in Appendix E and `Reports/performance_report.md`; this chapter states only what it establishes.

6.1 Scope and Threat Model

FTAD is a single-user desktop application with no network layer, no server component and no multi-user access control. Stating that first prevents the chapter from grading the application against controls it was never built to have.

Assets: transaction records (account identifiers, amounts, timestamps) and investigation records (case status, assignee, reviewer notes).

Adversaries considered: another local user with filesystem access; anyone who obtains the database file by backup, shared folder or stolen disk; and malicious input supplied through

CSV import.

Out of scope: remote network attackers (no network layer exists), physical access to an unlocked session, and supply-chain compromise of Qt or SQLite. An application with no network listener cannot be attacked remotely, and reporting "no firewall" would be noise rather than a finding.

6.2 What Was Examined

Area	Method	Result
SQL construction	Read all 12 `prepare()` sites; counted bindings	36 `addBindValue`, 0 interpolated queries
OS command execution	Search `QProcess`, `system`, `popen`, `exec*`	None
Network surface	Search socket and `QNetworkAccessManager` classes	None
Secrets in source	Search `password`, `secret`, `api_key`, `token`	None
Sensitive logging	Count `qDebug`/`qWarning`/`qCritical` in backend	0
Database constraints	Read `database/schema.sql`	`CHECK`, `NOT NULL`, `UNIQUE`, `FOREIGN KEY` present
File permissions	Inspect the created database file	`0644` — world-readable
Memory safety	ASan + UBSan, Valgrind (Chapter 5)	0 errors
Input validation	Read validators; run negative tests	Rejects malformed identifiers, amounts, timestamps

6.3 Findings

ID	Finding	Severity	Evidence	Status
SEC-01	Database created world-readable (0644)	Medium	`ls -l`; no `setPermissions` in source	Open
SEC-02	No authentication or authorisation	Medium (shared) / Low (single user)	Source review	Accepted by design
SEC-03	No encryption at rest; SQLite stores plaintext	Medium	`QSQLITE` driver, no SQLCipher	Accepted risk
SEC-04	Database stored beside the executable	Low	`DatabaseManager.cpp`	Open
SEC-05	No tamper detection on stored records	Low	Schema has no signature or hash chain	Accepted
SEC-06	Dependency vulnerability scanning not performed	Unknown	—	Gap, declared
SEC-07	CSV formula injection unguarded (latent)	Low	No export exists yet	Forward risk

6.4 Detailed Findings

6.4.1 SEC-01 — world-readable database

```
-rw-r--r-- 1 hashcat hashcat 57344 Nov  4 01:39 ftad.db
```

Any local user can read every transaction and every investigation note. On a single-user laptop this is inconsequential; on a shared machine it is a confidentiality breach requiring no exploit, only `cat`.

No code sets permissions — the file inherits the process umask, 022 on most Linux systems. The intent was clearly to keep the data private and a default silently did not. Remediation, two lines after the connection opens:

```
QFile::setPermissions(databaseFolder + "/ftad.db",
    QFile::ReadOwner | QFile::WriteOwner);
```

This is the only finding in the chapter that is a defect rather than a design consequence.

6.4.2 Accepted risks and forward risks

SEC-02, SEC-03, SEC-04, SEC-05 and SEC-07 are recorded in full in Appendix F, because each is a design consequence or a forward risk rather than a defect, and the detail is consulted rather than read. In summary: there is no authentication, no authorisation model and no encryption at rest, so whoever can launch the application can read and modify everything and anyone holding the database file can read it without the application; the database is resolved beside the executable rather than under `QStandardPaths::AppDataLocation`, which fails or requires elevation when installed system-wide; nothing detects direct edits to the database, since the schema enforces internal consistency but consistency is not authenticity; and CSV formula injection is unguarded, which is currently unreal because `ReportsPage` exports nothing but will become real the moment export exists.

6.5 What Was Not Found, and Why That Is Credible

A report listing only findings invites the question of whether the absences were tested or assumed. Each absence below was actively checked, and each is a search that could have returned a hit.

SQL injection — not present. Every query uses `prepare()` with `?` placeholders and `addBindValue()`: twelve prepared statements, 36 bind calls, and a search for `exec(... .arg(...))` returns nothing. This is the control most often broken in

Qt applications and here it is correct throughout. OS command injection — not present. No `QProcess`, `system()`, `popen()` or `exec*()` appears in `backend/` or `frontend/`; there is no shell to inject into. Memory safety — verified clean. ASan, UBSan and Valgrind report zero errors across 154 tests (§5.4), with 0 bytes definitely or indirectly lost. Secrets in source — none. No password, API key, token or credential string appears in backend, frontend or schema. Sensitive logging — none. The backend contains zero `qDebug`, `qWarning` or `qCritical` calls, so transaction data cannot leak into a log, terminal or crash report through application logging. Network attack surface — none exists. No socket or network class is used; this is not a control that was implemented but an architectural property that removes an entire class of risk. Input validation — present and tested. Identifiers are normalised and rejected when malformed, amounts must be finite and non-negative, timestamps must parse, and CSV rows with missing, duplicated or ill-typed columns are rejected with per-row errors — verified by 16 `CsvImportTest` and 18 `TransactionRepositoryTest` cases, not asserted from reading the code.

SEC-06 is a genuine gap, declared as one. No dependency vulnerability scanning was performed: Qt 6.8.2, SQLite 3.46.1 and GoogleTest 1.16.0 were not checked against CVE databases. Claiming "no known vulnerabilities" would require that scan, so no such claim is made.

6.6 Performance Analysis

Measured with `ftad_benchmark` at 1,000–20,000 transactions, median of five repetitions; the per-size table, the constructor comparison and the full scaling discussion are in Appendix E.

6.6.1 The defect that dominated everything

Timestamps were parsed with `QDateTime::fromString` and stored as local time. Constructing a local `QDateTime` resolves the system timezone per record, and at 20,000 records that dominated the entire analysis: the same date and time constructed as `Qt::LocalTime` cost 2,901 ms, against 6 ms for `Qt::UTC` — a factor of roughly 500 on the constructor alone.

The fix introduced `backend/include/TransactionTime.h`, which builds the instant explicitly in UTC. End-to-end analysis at 20,000 transactions fell from 40.50 s to 27.6 ms, roughly 1,466×. This is the clearest example in the project of a performance de-

fect invisible in code review: the call is idiomatic, the result is correct, and only measurement reveals what it costs.

6.6.2 Current baseline and its noise

At 20,000 transactions the current build analyses in 31.7–39.9 ms, median 34.8 ms, with `build()` taking 12.87 ms. Those figures must be read as a range: three runs of the same binary on the same data spanned roughly 25 %, because no CPU frequency scaling was pinned and other processes were running. The report therefore quotes the range and the median and draws no conclusion from differences smaller than the spread; the historical 27.6 ms belongs to its own run and is not blended with these.

Per-transaction cost stays between 1.7 and 2.0 µs across the whole range, so the algorithm is effectively linear. The measured scaling factor at 20,000 is 23.5× against a linear ideal of 20×; the excess is consistent with cache effects and with the noise above, and the per-transaction column — which would rise steadily under quadratic behaviour — does not rise.

6.7 Defect Summary

ID	Defect	Severity	Found by	Status
BUG-01	Behaviour baseline included future transactions	High	`BehaviorAnalyzer` tests	Fixed
BUG-02	Investigation cases destroyed by re-analysis	High	`DetectionPipelineTest`	Fixed
BUG-03	Uncommitted SQLite transaction on no-op save	High	`InvestigationTest`	Fixed
BUG-04	Duplicate and malformed identifiers accepted	High	`TransactionRepositoryTest`, `CsvImportTest`	Fixed
BUG-05	Sample dataset did not exercise the detection rules	Medium	Manual review + detection tests	Fixed, verified end to end
BUG-06	Analysis runtime dominated by local-time construction	Medium	`ftad_benchmark`	Fixed (~1,466×)
LAT-01	Implicit `int` multiplication widened to `qint64`	Low	Clang-Tidy	Open, unreachable
LAT-02	`minimumHistory` read as >= and > in two methods	Low	Boundary analysis	Open, dead code
SEC-01	Database file world-readable	Medium	File inspection	Open
SEC-03	No encryption at rest	Medium	Driver and schema review	Accepted risk

Six defects found and fixed; four issues open and documented with their reachability and remedies. None of the open items affects detection correctness in the shipped configuration.

6.8 Recommendations

Ordered by value per unit of effort. Setting the database permissions to 0600 after creation addresses SEC-01 in two lines. Adding upper bounds to both window settings in `validate()` and casting the window arithmetic to `qint64` address LAT-01 in five lines between them. Making `hasBaseline()` and `profile()` agree with `evaluate()` on `minimumHistory` is a one-comparison change and closes LAT-02. Those four together are under ten lines and close four of the five open items. Moving the database to `QStandardPaths::AppDataLocation` addresses SEC-04 and is a small change; prefixing formula-leading characters on CSV export addresses SEC-07 and is small but only becomes necessary once export exists; running dependency vulnerability scanning in CI addresses SEC-06 and is an ongoing commitment rather than a change; and encrypting at rest addresses SEC-03 and is a large piece of work that should be undertaken only if the application is deployed with real financial data.

Recommendations 1–4 were not applied because doing so would have invalidated the measurements in Chapter 5, which were taken against the current commit. Applying them and re-running the gates is the correct next action, not a late edit to a measured baseline.

6.9 Conclusion

The security posture is what the architecture implies: a single-user local application with no network surface, correct SQL parameterisation, clean memory behaviour, no secrets and no sensitive logging — and no authentication, no encryption, and a database any local user can read.

The two findings that matter are SEC-01 and SEC-03, and they differ in kind. SEC-01 is a defect: the intent was privacy and a default silently overrode it. SEC-03 is a scope decision: encryption was never attempted, and the application is honest about that only if the report is.

The performance work produced the largest measurable improvement in the project — 40.50 s to 27.6 ms — from a defect no code review would have caught. That is the argument for measurement as a testing activity rather than an optional extra.

What this chapter does not establish is that FTAD is secure in an absolute sense. It establishes that specific named properties were checked, that most hold, and that the ones that do not

are listed with their consequences.

Chapter 7 — Installation Manual

> Evidence note. The procedure below was executed end to end in a clean directory as part of this report. The transcript is `documentation/evidence/install/install-verify-20261104-055026.log`, produced by `./verify_install.sh`: 14 of 14 checks passed, zero compiler warnings, 46 s build. Every command in this chapter appears in that transcript.

7.1 Scope

FTAD is a desktop application built with C++17 and Qt 6. It has no installer package, no service component and no network dependency: it is compiled from source and run as a single executable. This chapter covers installing the build dependencies, compiling, verifying, launching, and removing the application.

The supported platform is 64-bit Linux with a Debian-style package manager. The procedure was verified on one such system and on no other. Windows and macOS are not claimed to work: Qt and CMake are portable, but the launch scripts in §7.5 are POSIX shell and the tested toolchain is GCC on Linux.

7.2 Requirements

Component	Minimum	Verified with	Purpose
C++ compiler	C++17	GCC 14.2.0	Builds all sources
CMake	3.16	3.31.6	Build system (`CMakeLists.txt`)
Qt 6 Base	6.x	6.8.2	`Qt6::Core`, `Qt6::Widgets`, `Qt6::Sql`
Qt 6 Charts	6.x	6.8.2	`Qt6::Charts`, linked by the GUI target
Build tool	—	Ninja	Drives the compile
SQLite CLI	optional	3.46.1	Inspecting the database by hand

Qt 6 Charts is a separate package from Qt 6 Base and is easy to miss, because the backend library does not need it — only the `FTAD` executable does, so an absent Charts package fails the build late, at link time, rather than at configure time. A Release build needs roughly

120 MB of build tree; the test suite pulls GoogleTest through CMake's `FetchContent`, which needs network access on the first configure only, and adds about 60 MB.

7.3 Installing the dependencies

```
sudo apt update
sudo apt install build-essential cmake ninja-build
 ↪  qt6-base-dev qt6-charts-dev
```

`build-essential` supplies GCC, `make` and the C library headers. `ninja-build` is optional — CMake falls back to `make` — but shortens the build considerably. Verify before continuing with `cmake --version` (3.16 or newer), `g++ -dumpversion` (11 or newer) and `pkg-config --modversion Qt6Core Qt6Charts Qt6Sql`. The last should print three version numbers; if it prints `Package Qt6Charts was not found`, the Charts package is missing and the GUI will not link.

7.4 Building from source

From the project root — the directory containing `CMakeLists.txt`:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j"$(nproc)"
```

The first command configures, the second compiles. On the verified system the build took 46 s on all cores and produced `build/FTAD` (420 KB). Two CMake options control what is built: `FTAD_BUILD_TESTS` (default `ON`) builds `tests/ftad_tests` and registers it with CTest, and `FTAD_ENABLE_COVERAGE` (default `OFF`) adds `--coverage -O0 -g` to `ftad_core`. A normal installation wants the application alone, which is `cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DFTAD_BUILD_TESTS=OFF` followed by the same build command.

Configuration also copies `database/schema.sql` into `build/database/`. This is not cosmetic: `DatabaseManager` reads the schema from beside the executable at first launch, so a build tree missing that file fails immediately with `Unable to apply the database schema.`

7.5 First run, verification and launching

```
./build/FTAD
```

On first launch the application creates `build/database/ftad.db`, applies

the four-table schema (`transactions`, `anomalies`, `investigations`, `investigation_events`) and enables foreign-key enforcement with `PRAGMA foreign_keys = ON`. There is no separate initialisation step and no migration tool — deleting the file returns the application to a clean state. Confirm the installation with `sqlite3 build/database/ftad.db ".tables"`, which should print `anomalies investigation_events investigations transactions`. Then import the shipped sample from the Transactions page (Import CSV) and select `data/samples/test_transactions.csv`: eleven transactions are imported, of which four are flagged on analysis — one large amount with a behavioural deviation, one duplicate pair, and one rapid-withdrawal sequence. This is the quickest end-to-end confirmation that detection is working.

One caveat worth knowing before it surprises you. The database is created with mode 0644 — world-readable — because no code sets the permissions and the file inherits the process umask. This is finding SEC-01 in Chapter 6. On a single-user machine it is inconsequential; on a shared machine, run `chmod 600 build/database/ftad.db`.

Two convenience launchers sit at the project root. `./run.sh` starts the permanently built application, and `ftad.desktop` is a freedesktop entry for a menu or taskbar, installed with `install -Dm755 ftad.desktop ~/.local/share/applications/ftad.desktop`. Both contain an absolute `Exec` path pointing at `/home/hashcat/Documents/dataset/FTAD-main/build/FTAD`. If you build somewhere else, edit that path in both files, or the shortcut fails silently — a desktop launcher gives no error when its target is missing.

7.6 Running the test suite

```
cd build && QT_QPA_PLATFORM=offscreen ctest --output-on-failure
```

The suite is 154 tests across ten binaries and completes in a few seconds. The `QT_QPA_PLATFORM=offscreen` setting matters on a headless machine: without it Qt aborts when it cannot open a display, and the failure looks like a test failure rather than a missing display server. To regenerate every piece of evidence in this report — tests, coverage, static analysis, sanitizers, Valgrind and the benchmark — run `./run_test_gates.sh` (about four minutes), which writes `documentation/evidence/SUMMARY.md` and the logs beside it. The optional tools it needs are

installed by `./install_test_tools.sh`; check without installing using `./install_test_tools.sh --check`.

Coverage and sanitizer builds need their own trees, because optimisation distorts line attribution and instrumenting the Release tree yields misleading percentages: `cmake -S . -B build/coverage -DCMAKE_BUILD_TYPE=Debug -DFTAD_ENABLE_COVERAGE=ON`, built and tested in that directory. The sanitizer flags are not a CMake option and are passed directly; the exact invocation is recorded in §5.2 and in `run_test_gates.sh`. Keeping the three trees separate is not stylistic — mixing them is the most common way to obtain a coverage figure that does not describe the code tested.

7.7 Removing the application

FTAD installs nothing outside its own directory:

```
rm -r build/                                    # binaries and
 ↪  database
rm ~/.local/share/applications/ftad.desktop     # if the
 ↪  shortcut was installed
```

Deleting `build/` deletes the database with it, so copy `build/database/ftad.db` first if the transaction data matters. The Qt and CMake packages from §7.3 are shared system libraries and should be removed only if nothing else needs them.

7.8 Troubleshooting

Symptom	Cause	Remedy
`Could not find a package configuration file provided by "Qt6"`	Qt 6 development files absent	`sudo apt install qt6-base-dev qt6-charts-dev`
Configure succeeds, link fails on `Qt6::Charts`	Charts package missing	`sudo apt install qt6-charts-dev`
`Unable to create database directory`	Build tree not writable, or installed under `/opt`	Build in a user-writable directory; see SEC-04
`Unable to apply the database schema`	`build/database/schema.sql` missing	Re-run the configure step
Tests abort with `could not connect to display`	No X server	Prefix with `QT_QPA_PLATFORM=offscreen`
Shortcut does nothing	Stale `Exec` path in `ftad.desktop`	Edit the path; see §7.5
`FetchContent` fails on first configure	No network access	Build with `-DFTAD_BUILD_TESTS=OFF`, or vendor GoogleTest

The procedure above is not transcribed from intent — it was executed. `./verify_install.sh` performs the whole sequence in a throwaway directory and asserts fourteen conditions: the six dependencies present, a Release configure with tests enabled, a build completing with zero compiler warnings, the `FTAD` binary produced, the schema deployed

with four tables, 154 of 154 tests passing, a binary that stays in its event loop for five seconds offscreen, a first-run database created, and a queryable `transactions` table. All fourteen passed. Run it yourself with `./verify_install.sh`, or `--quick` to skip the test suite; it needs no root, modifies no repository file, and writes its transcript to `documentation/evidence/install/`.

Chapter 8 — Conclusion

The testing campaign set out to establish whether FTAD implements its specification, and on the evidence gathered it does. All 154 automated tests pass, coverage reaches 85.4 % of lines and 97.4 % of functions, Cppcheck finds nothing in application source, and three independent dynamic tools — AddressSanitizer, UndefinedBehaviorSanitizer and Valgrind — report no memory or undefined-behaviour errors across the full suite. Six defects were found and fixed. Four of them were found by tests written for this campaign rather than during construction, which is the result worth carrying forward: the suite detected failures that building the software had not. The shipped sample dataset was shown to exercise all four detection rules end to end, with one transaction triggering two rules simultaneously and so exercising the additive scoring path. Performance work produced the largest single improvement in the project, from 40.50 s to 27.6 ms at 20,000 transactions, and it came from a defect that no amount of code review would have found, because the offending call was idiomatic and its result was correct.

The limits of that evidence matter as much as the evidence. Branch coverage is 51.4 %, so while most lines are executed, roughly half of all decision outcomes are never exercised; the line figure should not be read as a statement about decision coverage. The lowest coverage sits in persistence error paths that cannot be reached without injecting a fault. No labelled fraud corpus exists, so no precision, recall or accuracy figure is claimed anywhere in this report — the rules are shown to implement their definitions, not to identify real fraud. The graphical interface was assessed by inspection only, because no widget-level harness was built; `ReportsPage` and `SettingsPage` remain placeholders with no behaviour to test. Two latent defects remain open by choice, both unreachable in the shipped configu-

ration, and two security findings remain open: the database is created world-readable, and nothing is encrypted at rest. Everything was measured on one Linux host, so portability is declared rather than demonstrated.

Three actions follow. First, the open items should be closed and the gates re-run: setting the database to mode 0600, bounding the two window settings and making `hasBaseline()` and `profile()` agree with `evaluate()` on `minimumHistory` together amount to under ten lines of change, and they were deferred only because applying them would have invalidated a measured baseline. Second, branch coverage should be raised deliberately by adding negative-path tests to the repository methods, which is where the gap is concentrated. Third, if the detector is ever to be claimed as accurate, a labelled dataset must be obtained and precision and recall measured against it; until then the honest description of FTAD is a rule-based screening aid whose rules work as specified, not a fraud detector with demonstrated detection performance.

References

[1] G. J. Myers, C. Sandler, and T. Badgett, The Art of Software Testing, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, 2012. doi: 10.1002/9781119202486.

[2] P. Ammann and J. Offutt, Introduction to Software Testing, 2nd ed. Cambridge, UK: Cambridge University Press, 2016. doi: 10.1017/9781316771273.

[3] P. C. Jorgensen, Software Testing: A Craftsman's Approach, 4th ed. Boca Raton, FL, USA: Auerbach Publications, 2013. doi: 10.1201/9781439889503.

[4] B. Beizer, Software Testing Techniques, 2nd ed. New York, NY, USA: Van Nostrand Reinhold, 1990.

[5] C. Y. Laporte and A. April, Software Quality Assurance Hoboken, NJ, USA: John Wiley & Sons, 2017. doi: 10.1002/9781119312451.

[6] S. H. Kan, Metrics and Models in Software Quality Engineering, 2nd ed. Boston, MA, USA: Addison-Wesley, 2003.

[7] ISO/IEC, ISO/IEC 25010:2023 Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model, International Organization for Standardization, Geneva, Switzerland, 2023.

[8] A. Stefani, E. Vamvatsikou, and B. Vassiliadis, "Insights into B2C E-Commerce Quality Using ISO 25010," Journal of Software Engineering and Applications, vol. 16, no. 11, pp. 622–639, 2023. doi: 10.4236/jsea.2023.1611032.

[9] IEEE, IEEE Std 1012-2016 IEEE Standard for System, Software, and Hardware Verification and Validation, IEEE, New York, NY, USA, 2017. doi: 10.1109/IEEESTD.2017.8055462.

[10] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.

[11] H. Wu, C. Nie, J. Petke, Y. Jia, and M. Harman, "An Empirical Comparison of Combinatorial Testing, Random Testing and Adaptive Random Testing," IEEE Transactions on Software Engineering, vol. 46, no. 3, pp. 302–320, 2020. doi: 10.1109/TSE.2018.2852744.

[12] S. Liu and S. Nakajima, "Automatic Test Case and Test Oracle Generation Based on Functional Scenarios in Formal Specifications for Conformance Testing," IEEE Transactions on Software Engineering, vol. 48, no. 2, pp. 691–712, 2022. doi: 10.1109/TSE.2020.2999884.

[13] A. Aghamohammadi, S. Mirian-Hosseinabadi, and S. Jalali, "Statement frequency coverage: A code coverage criterion for assessing test suite effectiveness," Information and Software Technology, vol. 129, pp. 106426, 2021. doi: 10.1016/j.infsof.2020.106426.

[14] J. Lee, S. Kang, and P. Jung, "Test coverage criteria for software product line testing: Systematic literature review," Information and Software Technology, vol. 122, pp. 106272, 2020. doi: 10.1016/j.infsof.2020.106272.

[15] A. Hora, "Excluding code from test coverage: practices, motivations, and impact," Empirical Software Engineering, vol. 28, no. 1, 2022. doi: 10.1007/s10664-022-10259-7.

[16] P. Derakhshanfar, X. Devroey, and A. Zaidman, "Basic block coverage for search-based unit testing and crash reproduction," Empirical Software Engineering, vol. 27, no. 7, 2022. doi: 10.1007/s10664-022-10155-0.

[17] I. Forgács and A. Kovács, Modern Software Testing Techniques: A Practical Guide for Developers and Testers Berkeley, CA, USA: Apress, 2024. doi: 10.1007/978-1-4842-9893-0.

[18] M. Kessel and C. Atkinson, "Diversity-driven unit test generation," Journal of Systems and Software, vol. 193, pp. 111442, 2022. doi: 10.1016/j.jss.2022.111442.

[19] H. D. Menéndez, G. Jahangirova, F. Sarro, P. Tonella, and D. Clark, "Diversifying Focused Testing for Unit Testing," ACM Transactions on Software Engineering and Methodology, vol. 30, no. 4, pp. 1–24, 2021. doi: 10.1145/3447265.

[20] A. Arcuri and J. P. Galeotti, "Handling SQL Databases in Automated System Test Generation," ACM Transactions on Software Engineering and Methodology, vol. 29, no. 4, pp. 1–31, 2020. doi: 10.1145/3391533.

[21] R. Greca, B. Miranda, and A. Bertolino, "State of Practical Applicability of Regression Testing Research: A Live Systematic Literature Review," ACM Computing Surveys, vol. 55, no. 13s, pp. 1–36, 2023. doi: 10.1145/3579851.

[22] S. Mondal and R. Nasre, "Colosseum: Regression Test Prioritization by Delta Displacement in Test Coverage," IEEE Transactions on Software Engineering, vol. 48, no. 10, pp. 4060–4073, 2022. doi: 10.1109/TSE.2021.3111169.

[23] G. Rothermel and R. H. Untch, On "Prioritizing Test Cases for Regression Testing" IEEE Transactions on Software Engineering, vol. 51, no. 3, pp. 802–807, 2025. doi: 10.1109/TSE.2025.3538490.

[24] G. Maurina, W. Cazzola, and S. Ghosh, "BabelRTS: Polyglot Regression Test Selection," IEEE Transactions on Software Engineering, vol. 51, no. 5, pp. 1487–1499, 2025. doi: 10.1109/TSE.2025.3554403.

[25] A. Bertolino, B. Miranda, R. Pietrantuono, and S. Russo, "Adaptive Test Case Allocation, Selection and Generation Using Coverage Spectrum and Operational Profile," IEEE Transactions on Software Engineering, vol. 47, no. 5, pp. 881–898, 2021. doi:

10.1109/TSE.2019.2906187.

[26] I. Bluemke and A. Malanowska, "Software Testing Effort Estimation and Related Problems," ACM Computing Surveys, vol. 54, no. 3, pp. 1–38, 2021. doi: 10.1145/3442694.

[27] Google, "GoogleTest User's Guide," 2024. [Online]. Available: https://google.github.io/googletest/

[28] Cppcheck Project, "Cppcheck Manual," 2024. [Online]. Available: https://cppcheck.sourceforge.io/manual.pdf

[29] Valgrind Developers, "Valgrind User Manual," 2024. [Online]. Available: https://valgrind.org/docs/manual/manual.html

[30] LLVM Project, "AddressSanitizer and UndefinedBehaviorSanitizer Reference," 2024. [Online]. Available: https://clang.llvm.org/docs/AddressSanitizer.html

[31] Z. Chen, Q. Zhang, J. Wu, J. Yan, and J. Xue, "A Source-Level Instrumentation Framework for the Dynamic Analysis of Memory Safety," IEEE Transactions on Software Engineering, vol. 49, no. 4, pp. 2107–2127, 2023. doi: 10.1109/TSE.2022.3210580.

[32] N. U. Eisty and J. C. Carver, "Testing research software: a survey," Empirical Software Engineering, vol. 27, no. 6, 2022. doi: 10.1007/s10664-022-10184-9.

[33] G. Murazvu, S. Parkinson, S. Khan, N. Liu, and G. Allen, "A Survey on Factors Preventing the Adoption of Automated Software Testing: A Principal Component Analysis Approach," Software, vol. 3, no. 1, pp. 1–27, 2024. doi: 10.3390/software3010001.

[34] S. Stradowski and L. Madeyski, "Exploring the challenges in software testing of the 5G system at Nokia: A survey," Information and Software Technology, vol. 153, pp. 107067, 2023. doi: 10.1016/j.infsof.2022.107067.

[35] S. Fatima, H. Hemmati, and L. C. Briand, "FlakyFix: Using Large Language Models for Predicting Flaky Test Fix Categories and Test Code Repair," IEEE Transactions on Software Engineering, vol. 50, no. 12, pp. 3146–3171, 2024. doi: 10.1109/TSE.2024.3472476.

[36] M. Imran, V. Cortellessa, D. Di Ruscio, R. Rubei, and L. Traini, "Is code coverage of performance tests related to source code features? An empirical study on open-source Java systems," Empirical Software Engineering, vol. 30, no. 6, 2025. doi: 10.1007/s10664-025-10712-3.

[37] L. Traini and V. Cortellessa, "DeLag: Using Multi-Objective Optimization to Enhance the Detection of Latency Degradation Patterns in Service-Based Systems," IEEE Transactions on Software Engineering, 2023. doi: 10.1109/TSE.2023.3266041.

[38] P. Lachkov, L. Tawalbeh, and S. Bhatt, "Vulnerability Assessment for Applications Se-

curity Through Penetration Simulation and Testing," Journal of Web Engineering, 2022. doi: 10.13052/jwe1540-9589.2178.

[39] M. Leithner, B. Garn, and D. E. Simos, "HYDRA: Feedback-driven black-box exploitation of injection vulnerabilities," Information and Software Technology, vol. 140, pp. 106703, 2021. doi: 10.1016/j.infsof.2021.106703.

[40] Y. Zhao, K. Damevski, and H. Chen, "A Systematic Survey of Just-in-Time Software Defect Prediction," ACM Computing Surveys, vol. 55, no. 10, pp. 1–35, 2023. doi: 10.1145/3567550.

[41] G. E. Cabral and L. L. Minku, "Towards Reliable Online Just-in-Time Software Defect Prediction," IEEE Transactions on Software Engineering, vol. 49, no. 3, pp. 1342–1358, 2023. doi: 10.1109/TSE.2022.3175789.

[42] S. Herbold, "On the Costs and Profit of Software Defect Prediction," IEEE Transactions on Software Engineering, vol. 47, no. 11, pp. 2617–2631, 2021. doi: 10.1109/TSE.2019.2957794.

[43] J. Yao and M. Shepperd, "The impact of using biased performance metrics on software defect prediction research," Information and Software Technology, vol. 139, pp. 106664, 2021. doi: 10.1016/j.infsof.2021.106664.

[44] S. Stradowski and L. Madeyski, "Machine learning in software defect prediction: A business-driven systematic mapping study," Information and Software Technology, vol. 155, pp. 107128, 2023. doi: 10.1016/j.infsof.2022.107128.

[45] H. Jahanshahi and M. Cevik, "S-DABT: Schedule and Dependency-aware Bug Triage in open-source bug tracking systems," Information and Software Technology, vol. 151, pp. 107025, 2022. doi: 10.1016/j.infsof.2022.107025.

[46] W. Hilal, S. A. Gadsden, and J. Yawney, "Financial Fraud: A Review of Anomaly Detection Techniques and Recent Advances," Expert Systems with Applications, vol. 193, pp. 116429, 2022. doi: 10.1016/j.eswa.2021.116429.

[47] T. Pourhabibi, K. Ong, B. H. Kam, and Y. L. Boo, "Fraud detection: A systematic literature review of graph-based anomaly detection approaches," Decision Support Systems, vol. 133, pp. 113303, 2020. doi: 10.1016/j.dss.2020.113303.

[48] S. Motie and B. Raahemi, "Financial fraud detection using graph neural networks: A systematic review," Expert Systems with Applications, vol. 240, pp. 122156, 2024. doi: 10.1016/j.eswa.2023.122156.

[49] B. Baesens, S. Höppner, and T. Verdonck, "Data engineering for fraud detection," Deci-

sion Support Systems, vol. 150, pp. 113492, 2021. doi: 10.1016/j.dss.2021.113492.

[50] K. Lin and Y. Gao, "Model interpretability of financial fraud detection by group SHAP," Expert Systems with Applications, vol. 210, pp. 118354, 2022. doi: 10.1016/j.eswa.2022.118354.

[51] ISO/IEC/IEEE, ISO/IEC/IEEE 29119-2:2021 Software and systems engineering — Software testing — Part 2: Test processes, International Organization for Standardization, Geneva, Switzerland, 2021.

[52] ISO/IEC/IEEE, ISO/IEC/IEEE 29119-3:2021 Software and systems engineering — Software testing — Part 3: Test documentation, International Organization for Standardization, Geneva, Switzerland, 2021.

Appendix A — Reference Verification Record

Verification date: 4 November 2026 Sources of truth:

- the Crossref REST API (`api.crossref.org`), for journal articles and books;
- the Open Library catalogue (`openlibrary.org`), for ISBNs and classic textbooks;
- a direct HTTP fetch of each tool documentation URL.

Method

For each journal article and book, the DOI (or ISBN) was submitted to the Crossref registration record or the Open Library catalogue and the returned metadata — title, author list, container, year, volume, issue, pages — was transcribed into `references.bib`. Nothing was written from memory. Where a DOI was unavailable (classic textbooks), the ISBN was verified through Open Library.

For tool documentation, each URL was fetched and required to return HTTP 200.

For ISO/IEC and IEEE standards, `iso.org` returns HTTP 403 to automated clients, so the standards are cited by standard number only and are flagged below as not machine-verified.

Summary

Category	Entries	Verified	Unverified
Books	8	8	0
Journal articles	36	36	0
Standards	4	0	4
Tool documentation	4	4	0
Total	52	48	4

Academic share (books + journals): 44 / 52 = 84.6 %.

Books

Key	Title	Year	DOI / ISBN	Verified via
myers2012art	The Art of Software Testing (3rd ed.)	2012	10.1002/9781119202486	Crossref
ammann2016introduction	Introduction to Software Testing (2nd ed.)	2016	10.1017/9781316771273	Crossref
jorgensen2013software	Software Testing: A Craftsman's Approach (4th ed.)	2013	10.1201/9781439889503	Crossref
laporte2017sqa	Software Quality Assurance	2017	10.1002/9781119312451	Crossref
forgacs2024modern	Modern Software Testing Techniques	2024	10.1007/978-1-4842-9893-0	Crossref
kan2003metrics	Metrics and Models in Software Quality Engineering (2nd ed.)	2003	9780201729153	Open Library
beizer1990software	Software Testing Techniques (2nd ed.)	1990	9780442206727	Open Library
pressman2020software	Software Engineering: A Practitioner's Approach (9th ed.)	2020	9781259872976	Open Library

Journal articles

#	Key	Journal	Year	DOI
1	mondal2022colosseum	IEEE TSE	2022	10.1109/TSE.2021.3111169
2	liu2022automatic	IEEE TSE	2022	10.1109/TSE.2020.2999884
3	wu2020empirical	IEEE TSE	2020	10.1109/TSE.2018.2852744
4	rothermel2025prioritizing	IEEE TSE	2025	10.1109/TSE.2025.3538490
5	bertolino2021adaptive	IEEE TSE	2021	10.1109/TSE.2019.2906187
6	fatima2024flakyfix	IEEE TSE	2024	10.1109/TSE.2024.3472476
7	maurina2025babelrts	IEEE TSE	2025	10.1109/TSE.2025.3554403
8	kessel2022diversity	J. Syst. Softw.	2022	10.1016/j.jss.2022.111442
9	menendez2021diversifying	ACM TOSEM	2021	10.1145/3447265
10	greca2023state	ACM CSUR	2023	10.1145/3579851
11	bluemke2021software	ACM CSUR	2021	10.1145/3442694
12	aghamohammadi2021statement	Inf. Softw. Technol.	2021	10.1016/j.infsof.2020.106426
13	lee2020test	Inf. Softw. Technol.	2020	10.1016/j.infsof.2020.106272
14	hora2022excluding	Empir. Softw. Eng.	2022	10.1007/s10664-022-10259-7
15	derakhshanfar2022basic	Empir. Softw. Eng.	2022	10.1007/s10664-022-10155-0
16	chen2023source	IEEE TSE	2023	10.1109/TSE.2022.3210580
17	arcuri2020handling	ACM TOSEM	2020	10.1145/3391533
18	herbold2021costs	IEEE TSE	2021	10.1109/TSE.2019.2957794
19	cabral2023towards	IEEE TSE	2023	10.1109/TSE.2022.3175789
20	zhao2023systematic	ACM CSUR	2023	10.1145/3567550
21	yao2021impact	Inf. Softw. Technol.	2021	10.1016/j.infsof.2021.106664
22	stradowski2023machine	Inf. Softw. Technol.	2023	10.1016/j.infsof.2022.107128
23	jahanshahi2022sdabt	Inf. Softw. Technol.	2022	10.1016/j.infsof.2022.107025
24	eisty2022testing	Empir. Softw. Eng.	2022	10.1007/s10664-022-10184-9
25	murazvu2024survey	Software	2024	10.3390/software3010001
26	stradowski2023exploring	Inf. Softw. Technol.	2023	10.1016/j.infsof.2022.107067
27	imran2025code	Empir. Softw. Eng.	2025	10.1007/s10664-025-10712-3
28	traini2023delag	IEEE TSE	2023	10.1109/TSE.2023.3266041
29	leithner2021hydra	Inf. Softw. Technol.	2021	10.1016/j.infsof.2021.106703
30	lachkov2022vulnerability	J. Web Eng.	2022	10.13052/jwe1540-9589.2178
31	hilal2022financial	Expert Syst. Appl.	2022	10.1016/j.eswa.2021.116429
32	pourhabibi2020fraud	Decis. Support Syst.	2020	10.1016/j.dss.2020.113303
33	baesens2021data	Decis. Support Syst.	2021	10.1016/j.dss.2021.113492
34	motie2024financial	Expert Syst. Appl.	2024	10.1016/j.eswa.2023.122156

#	Key	Journal	Year	DOI
35	lin2022model	Expert Syst. Appl.	2022	10.1016/j.eswa.2022.118354
36	stefani2023insights	J. Softw. Eng. Appl.	2023	10.4236/jsea.2023.1611032

Tool documentation (HTTP 200 on 2026-11-04)

Key	URL	Status
googletest	https://google.github.io/googletest/	200
cppcheck	https://cppcheck.sourceforge.io/manual.pdf	200
valgrind	https://valgrind.org/docs/manual/manual.html	200
clangsanitizers	https://clang.llvm.org/docs/AddressSanitizer.html	200

Standards — NOT machine-verified

`iso.org` returns HTTP 403 to automated clients. These four entries are cited by standard number and must be checked manually against the university library catalogue or the ISO/IEC web store before submission. Their existence and numbering are widely attested, but this record does not claim to have verified them.

Key	Standard
iso29119_2	ISO/IEC/IEEE 29119-2:2021 — Software testing, Part 2: Test processes
iso29119_3	ISO/IEC/IEEE 29119-3:2021 — Software testing, Part 3: Test documentation
iso25010	ISO/IEC 25010:2023 — SQuaRE, Product quality model
ieee1012	IEEE Std 1012-2016 — System, Software, and Hardware V&V

Caveats

1. Year attribution. Crossref's `issued` date is the registration date and can precede the print issue by several months for journals that publish online-first. Years in `references.bib` follow the Crossref record. Where an article appeared online in one year and in print in the next, the difference is at most one year and does not affect the recency analysis materially.
2. Missing page numbers. Four Empirical Software Engineering entries carry no page range because the publisher assigns article numbers rather than page ranges and Crossref returned none. Pages were not invented to fill the gap.
3. Recency interpretation. The lecturer's "three-year requirement" for journal references is ambiguous. This project interprets it as a preference for recent work and

reports the actual distribution (24/36 published 2022 or later) rather than asserting compliance with an unclear rule.

Appendix B — Test Case Design and Requirements Traceability

This appendix holds the test-case design and the requirements traceability matrix. They are placed here rather than in Chapter 4 because they are consulted rather than read: a reader checks one case identifier or one requirement row, and does not need them in the narrative flow.

The design decisions they embody are stated in Chapter 4 §4.6 and §4.11. In summary: cases are derived from the threshold values in §4.6.1, not from observed program behaviour, so that a test asserts the specification rather than restating the implementation. Every functional requirement is traced to at least one suite in B.6, and the requirements that cannot be verified are listed explicitly in B.6.3 rather than left to inference.

B.1 Test case identifier scheme

```
TC-<AREA>-<NNN>

AREA in { CSV, TXN, ID, DUP, LARGE, RAPID, BEH, RISK,
          SET, INV, TIME, SYS }
```

Examples: `TC-DUP-004`, `TC-SET-011`, `TC-INV-007`. The area token makes the traceability matrix readable without a lookup, and the numeric token is assigned in creation order and never reused, so a deleted case leaves a visible gap rather than silently renumbering.

B.2 Test case template

Every case records the ten fields the assignment specifies, plus the technique that generated it — without which the case cannot be defended as designed rather than discovered.

Field	Content
Test Case ID	`TC-<AREA>-<NNN>`
Objective	The single property being verified
Module	Backend class or suite under test
Technique	Design technique that produced the inputs

Field	Content
Preconditions	Fixture state required before step 1
Test data	Concrete values, including units
Steps	Ordered actions
Expected result	Assertion, stated before execution
Actual result	Observed outcome, recorded after execution
Status	Pass / Fail / Blocked / Not Executed
Evidence	Log file or CTest test name

B.3 Worked example: boundary value analysis on the large-amount rule

`largeAmountAbsolute` defaults to 10000.0 and `validate()` requires it to be strictly greater than zero and finite. The rule fires when a transaction amount exceeds the threshold — the comparison in `checkLargeAmount()` is `amount <= threshold → return`, so the operator is strict:

```
if (transaction.amount <= settings.largeAmountAbsolute)
    return;
```

Equivalence classes for amount, given a 10000.0 threshold:

Class	Range	Expected
Invalid	amount < 0	Rejected by repository validation
Below threshold	0 ≤ amount ≤ 10000.0	Rule does not fire
Above threshold	amount > 10000.0	Rule fires

Boundary set derived from those classes:

Case	Amount (LSL)	Rationale	Expected	Implemented by
`TC-LARGE-001`	9999.99	Largest value below threshold	Not flagged	`AmountExactly-AtTheLimitIs-NotLarge` (lower case)
`TC-LARGE-002`	10000.00	Exactly at threshold	Not flagged — > is strict	`AmountExactly-AtTheLimitIs-NotLarge`
`TC-LARGE-003`	10000.01	Smallest value above threshold	Flagged, weight 35.0	`LargeAmountIs-Flagged-WithoutHistory`
`TC-LARGE-004`	0.00	Lower bound of the valid amount range	Not flagged; accepted by schema	`RejectsZero-AndNegative-Amounts`
`TC-LARGE-005`	−0.01	Just below the valid range	Rejected by validation	`RejectsZero-AndNegative-Amounts`

`TC-LARGE-002` is the case that matters, and it is worth being precise about why. A boundary test exists to pin the operator, not to assert a preferred operator. This project's specification is "exceeds the limit", the reason text says "exceeds", and the code uses >; the test asserts that 10000.00 is not flagged. Had the test instead asserted that it is flagged,

it would have been testing a different specification from the one implemented — and the mismatch would have been a defect in the test, not in the code.

Boundary analysis of the other thresholds. These were checked against the source rather than assumed, and one ambiguity was found:

Parameter	Boundary cases	Operator in code	Result
`rapidWithdrawalCount` = 3	2, 3, 4 withdrawals in 60 min	`withdrawals < count → return`	Inclusive: 3 fires, 2 does not
`unusualAmountZScore` = 3.0	2.99, 3.00, 3.01	`zScore >= threshold && amount > mean`	Inclusive at 3.00
`minimumHistory` = 5	4, 5, 6 prior transactions	Two different operators	Ambiguous — see below
`zeroVarianceRatio` = 2.0	setting 1.00 (invalid), 1.01 (valid)	`validate()` requires > `1.0`	2.0 is valid; the open bound is at 1.0
`highThreshold` = 65.0	64.99, 65.00	band assignment is >=	65.00 bands as high

The ambiguity found. `minimumHistory` is read with two different operators in the same class. `evaluate()` accepts a baseline at exactly the threshold:

```
if (historyCount < minimumHistory)    // >= 5 permits a baseline
    return;
```

while `hasBaseline()` and `profile()` require strictly more:

```
return it->count > minimumHistory;    // > 5, so exactly 5
 ↪  reports no baseline
```

An account with exactly five prior transactions is therefore evaluated against a behavioural baseline while the profile simultaneously reports that no baseline exists. **Impact: none at run time.** Neither `hasBaseline()` nor `profile()` is called anywhere outside `BehaviorAnalyzer`; they are unused public API, and the detection path goes through `evaluate()` only. The finding is recorded as a code quality issue — the same setting should not mean two things — rather than as a defect, because nothing observable depends on the second meaning.

This is the most useful outcome the boundary technique produced: not a failing test, but a demonstration that the technique finds specification ambiguity as well as off-by-one errors. An equivalence-partition-only approach would have tested 4 and 6 prior transactions and missed the case at exactly 5.

B.4 Worked example: state transition on investigation status

The investigation lifecycle has four statuses and an asymmetric transition rule: resolving or dismissing closes the case, and moving back to an open status reopens it.

From	To	Expected	Case
pending	in_progress	Allowed; event recorded	TC-INV-002
in_progress	resolved	Allowed; `closed_at` set	TC-INV-003
resolved	in_progress	Allowed; `closed_at` cleared (reopen)	TC-INV-005
pending	resolved	Allowed	TC-INV-004
any	`invalid_status`	Rejected by `isValidStatus`	TC-INV-009

The reopen case is the interesting one: it asserts that a closed case can be reopened and that the audit trail retains the earlier closure rather than overwriting it. A state-transition approach finds this because it enumerates transitions rather than exercising a happy path.

B.5 Negative and robustness cases

Negative testing is not confined to validation suites. The following classes are covered explicitly:

Class	Example	Suite
Empty input	CSV with header only	`CsvImportTest`
Missing field	Row with 6 of 8 columns	`CsvImportTest`
Wrong type	Non-numeric amount	`CsvImportTest`
Duplicate record	Same `transaction_id` twice	`TransactionRepository-Test`
Malformed quoting	Unbalanced quote in a field	`CsvImportTest`
Blank lines	Trailing newlines and empty rows	`CsvImportTest`
Duplicate header	Two `amount` columns	`CsvImportTest`
Out-of-range setting	`rapidWithdrawalCount = 1`	`DetectionSettings`
Illegal state transition	Unknown case status	`InvestigationTest`
Invalid timestamp	Unparseable date string	`TransactionTime`

B.6 Requirements traceability matrix

Every functional requirement (FR) and non-functional requirement (NFR) is traced to the suite or tool that verifies it. A requirement with no test is a requirement with no evidence, and is flagged as such.

B.6.1 Functional requirements

ID	Requirement	Verified by	Tests
FR-01	Import transactions from a CSV file, reporting per-row errors	`CsvImportTest`	16
FR-02	Add a transaction manually through the repository	`Transaction-RepositoryTest`	18
FR-03	Validate and normalise account and transaction identifiers	`Identifiers`, `Transaction-RepositoryTest`	14 + 18
FR-04	Detect duplicate payments within a time window and tolerance	`AnomalyDetector`	20

ID	Requirement	Verified by	Tests
FR-05	Detect amounts at or above the large-amount threshold	`AnomalyDetector`	20
FR-06	Detect rapid successive withdrawals within a window	`AnomalyDetector`	20
FR-07	Detect behavioural deviation against a strictly-earlier account baseline	`BehaviorAnalyzer`, `AnomalyDetector`	13 + 20
FR-08	Score risk by weight and band into low / medium / high	`RiskScorer`, `AnomalyDetector`	10 + 20
FR-09	Persist detection results idempotently across re-analysis	`DetectionPipeline-Test`	13
FR-10	Create and update investigation cases	`InvestigationTest`	18
FR-11	Record an append-only audit history of case changes	`InvestigationTest`	18
FR-12	Validate detection settings against documented ranges	`DetectionSettings`	19

B.6.2 Non-functional requirements

ID	Requirement	Verified by	Status
NFR-01	Analyse 20,000 transactions within a usable interactive time	`ftad_benchmark`	Measured: 31.7–39.9 ms across three runs (median 34.8 ms)
NFR-02	No memory errors or undefined behaviour under test	ASan + UBSan; Valgrind	Verified — 0 findings in both
NFR-03	Referential integrity enforced across all four tables	`Transaction-RepositoryTest`, `InvestigationTest`	Verified
NFR-04	Test execution is deterministic and order-independent	Fixture design; full-suite runs	Verified (154/154; 3.8–5.4 s across runs)
NFR-05	Builds and runs on Linux	Build and launch on Parrot 7.4 / Debian 13	Verified on one distribution
NFR-06	Static analysis free of high-severity findings	Cppcheck, Clang-Tidy	Verified — 0 application-scope Cppcheck findings; 6 Clang-Tidy diagnostics triaged (§4.10.4.1)

B.6.3 Requirements without direct coverage

Requirement	Gap	Disposition
GUI rendering correctness	No widget test harness	Manual acceptance only; declared in §4.9.2
`ReportsPage` behaviour	Page is a placeholder	No behaviour exists to test
`SettingsPage` persistence	Page is a placeholder	No behaviour exists to test
Multi-user concurrency	Not implemented	Out of scope
Detection accuracy (precision/recall)	No labelled ground-truth dataset	Not claimed anywhere in this report

The last row is the most important. The suite verifies that each rule implements its specification. It does not and cannot verify that the specification detects real fraud, because that would require labelled data the project does not possess. No accuracy figure is reported, and any claim that the detector is "accurate" would be unsupported.

B.7 Test design technique inventory

Ten techniques are used. The category column follows the usual black-box / white-box / experience-based / verification split; the "applied to" column names the artefact, not the suite, because one technique is often applied to several suites.

Technique	Category	Applied to	Rationale
Equivalence partitioning	Black-box	Identifier, settings and CSV field validation	Inputs cluster into classes with identical handling
Boundary value analysis	Black-box	Every numeric threshold in `DetectionSettings`	Off-by-one defects concentrate at class edges
Decision table	Black-box	Risk banding (medium ≥ 35, high ≥ 65) and weight combinations	Output depends on combinations of conditions
State transition	Black-box	Investigation status, including reopening	Legal and illegal transitions must both be covered
Error guessing	Experience-based	Malformed CSV, quoted fields, blank lines, duplicate headers/IDs	Cheap to apply, high yield on parser code
Statement coverage	White-box	`ftad_core` via gcov	Establishes which code the suite never reaches
Branch coverage	White-box	Conditionals in detector and validator	Statement coverage hides untaken branches
Static analysis	Verification	`ftad_core` and `frontend`	Detects classes testing may not trigger
Dynamic analysis	Verification	Test binary under sanitizers and Valgrind	Detects memory and UB faults at run time
Measurement	Performance	`AnomalyDetector`, `BehaviorAnalyzer`, `RiskScorer`	Establishes cost and scaling, not pass/fail

B.8 Threshold inventory and enforced ranges

Boundary value analysis is only as good as the boundaries it uses. These values are read from `DetectionSettings` and are the defaults the application runs with.

Parameter	Default	Range enforced by `validate()`
`largeAmountAbsolute`	10000.0	> 0, finite
`duplicateWindowMinutes`	10	≥ 1 (no upper bound)
`duplicateAmountTolerance`	0.01	≥ 0, finite
`rapidWithdrawalWindowMinutes`	60	≥ 1 (no upper bound)
`rapidWithdrawalCount`	3	≥ 2
`unusualAmountZScore`	3.0	> 0, finite
`zeroVarianceRatio`	2.0	> 1, finite
`minimumHistory`	5	≥ 1
`mediumThreshold`	35.0	> 0, finite
`highThreshold`	65.0	> mediumThreshold, ≤ 100

Several ranges are open (`> 0`, `> 1`) rather than closed, which changes the boundary set: for `largeAmountAbsolute` the invalid boundary is 0.0 itself and the valid boundary is the smallest representable value above it, not 1.0. The cases in B.3 respect this. The two missing upper bounds are not theoretical: they are what makes the latent defect described in §4.10.4.1 reachable.

B.9 Out-of-scope areas and reasons

Exclusions are stated so that the absence of tests is not mistaken for evidence of correctness.

Excluded	Reason
Qt widget rendering and layout	No GUI harness; needs a display and screenshot baseline
`ReportsPage`, `SettingsPage`	Placeholder implementations — no behaviour exists to test
Authentication, authorisation, encryption at rest	Not implemented
Network and multi-client concurrency	Single-user local application
Datasets above 20,000 transactions	Benchmark ceiling; beyond this is extrapolated, not measured
Localisation, platform matrix	Single-language UI; one Linux distribution tested

B.10 Entry and exit criteria with measured outcomes

Type	#	Criterion	Threshold	Result
Entry	E1	Source compiles with `-Wall -Wextra`, zero warnings	0	Met
Entry	E2	`ftad_core` links into both `FTAD` and `ftad_tests`	—	Met
Entry	E3	`schema.sql` present beside the test binary	—	Met
Entry	E4	Qt platform plugin resolves headless	—	Met
Entry	E5	Fixtures create and destroy a clean database	—	Met
Entry	E6	Requirements and acceptance conditions written down	—	Met (B.6)
Exit	X1	All automated tests pass	100 %	154/154 in 5.4 s
Exit	X2	No compiler warnings	0	0
Exit	X3	No sanitizer or Valgrind errors	0	0 — ASan+UBSan 154/154 clean; Valgrind 0 errors from 0 contexts
Exit	X4	Static analysis findings triaged	100 %	0 application-scope Cppcheck findings; 6 Clang-Tidy diagnostics, all triaged
Exit	X5	Line coverage recorded	≥ 80 %	85.4 % lines (992/1162), 97.4 % functions, 51.4 % branches
Exit	X6	Every requirement traced to a test	100 %	B.6
Exit	X7	Every raised defect has a status	100 %	BUG-01 … BUG-06 closed; LAT-01/02 open and unreachable
Exit	X8	Performance baseline recorded	5 sizes	1,000–20,000, 5 repetitions each

E6 is listed last but is logically first: without a written expectation there is nothing to test against, only behaviour to describe.

B.11 Risk register and threats to validity

ID	Risk	Likelihood	Impact	Mitigation
R1	Tests encode the implementation, not the specification	Medium	High	Derive expected values from `DetectionSettings` ranges, not observed output
R2	Coverage inflated by silently excluding code	Medium	Medium	State exclusions; report the raw figure
R3	Suite is non-deterministic and flaky	Low	High	Fresh DB per test, fixed timestamps, offscreen platform
R4	Tools unavailable, so gates cannot be evidenced	Realised, then closed	High	Installed 4 Nov 2026; all gates re-run with captured logs
R5	Performance measured on an unpinned, noisy machine	Realised	Medium	Report median of 5 repetitions and the observed range
R6	GUI untested, so presentation defects reach submission	High	Medium	Manual acceptance pass; declare the gap
R7	Single-distribution testing masks portability defects	Medium	Low	Declare the limitation; avoid portability claims

Threats to validity, distinct from project risks:

Threat	Effect	Mitigation applied	Residual
Author–tester identity	Tests share the designer's blind spots	Expected values derived from specification	Not eliminated
Single implementation	No differential testing against a reference	None	Accepted
No labelled data	Detection accuracy unmeasurable	Accuracy not claimed	Accepted
One platform	Portability untested	Limitation declared	Accepted
Unpinned hardware	Timing varies between runs	Medians plus reported range	Reduced
Placeholder pages	Two modules untested	Declared out of scope	Accepted

Appendix C — Test Case Catalogue

C.1 Relationship to the Automated Suite

The 154 automated tests are not 154 hand-written cases. They are the executable form of the 45 designed cases catalogued below, expanded where a case has several distinguishable variants. This appendix records the designed cases, the technique that produced each, the expected result stated before execution, and the test that implements it.

The distinction matters for review. A test that exists but was never designed is a test whose coverage cannot be argued; a case that was designed but never implemented is a gap that can be seen and counted. Every row below has both.

Case identifiers follow `TC-<AREA>-<NNN>`. The numeric token is assigned in creation order and never reused, so a deleted case leaves a visible gap rather than silently renumbering its successors.

Expected results are stated as the property to be demonstrated, not as the observed output. Where a case pins a comparison operator, the operator is named, because that is the part most likely to change silently.

C.2 Case Catalogue

ID	Objective	Technique	Expected result	Test	Status
`TC-DUP-001`	Duplicate payment flags both transactions	Decision table	Both TXNXXX1 and TXNXXX2 carry `duplicate_payment`	`DuplicatePaymentFlagsBothSides`	Pass
`TC-DUP-002`	Duplicate outside the time window is ignored	BVA	TXNXXX2 has no `duplicate_payment`	`DuplicateOutsideTheWindowIsNotFlagged`	Pass
`TC-DUP-003`	Duplicate outside the amount tolerance is ignored	BVA	TXNXXX2 has no `duplicate_payment`	`DuplicateOutsideAmountToleranceIsNotFlagged`	Pass
`TC-DUP-004`	Same amount to a different receiver is not a duplicate	Equivalence partitioning	No `duplicate_payment`	`DuplicateToADifferentReceiverIsNotFlagged`	Pass
`TC-LARGE-001`	Amount below the limit is not large	BVA	No `large_amount`	`AmountExactlyAtTheLimitIsNotLarge`	Pass

ID	Objective	Technique	Expected result	Test	Status
TC-LARGE-002	Amount exactly at the limit is not large	BVA	No large_amount — operator is strict >	AmountExactlyAtTheLimitIsNotLarge	Pass
TC-LARGE-003	Amount above the limit is large, without history	BVA	large_amount present, behaviour_deviation absent	LargeAmountIsFlaggedWithoutHistory	Pass
TC-RAPID-001	Third withdrawal in the window fires the rule	BVA	TXNXXX2 not flagged, TXNXXX3 flagged	RapidWithdrawalsAreFlaggedAtTheThreshold	Pass
TC-RAPID-002	Withdrawals outside the window do not fire	BVA	No rapid_withdrawals	WithdrawalsOutsideTheWindowAreNotRapid	Pass
TC-RAPID-003	Non-withdrawal types are never rapid withdrawals	Equivalence partitioning	No rapid_withdrawals	NonWithdrawalTypesAreNotRapidWithdrawals	Pass
TC-BEH-001	Insufficient history produces no behavioural finding	BVA	No behaviour_deviation	InsufficientHistoryProducesNoBehaviouralFinding	Pass
TC-BEH-002	Z-score deviation is reported with its magnitude	Decision table	behaviour_deviation present; reason contains "standard deviations"	BehaviouralDeviationReportsTheZScore	Pass
TC-BEH-003	Zero-variance history falls back to a ratio rule	Error guessing	behaviour_deviation present; reason contains "uniform"	UniformHistoryUsesTheRatioRule	Pass
TC-RISK-001	No rule hits means no score and no level	Equivalence partitioning	riskScore == 0.0, riskLevel empty	NoHitsMeansZeroScoreAndNoLevel	Pass
TC-RISK-002	A single hit scores its own weight	Decision table	riskScore == 40.0, level medium	SingleHitUsesItsWeight	Pass
TC-RISK-003	Multiple hits add up and band as high	Decision table	riskScore == 75.0, level high	MultipleHitsAddUp	Pass
TC-RISK-004	Score is capped at 100	BVA	riskScore == 100.0, level high	ScoreNeverExceedsOneHundred	Pass
TC-RISK-005	Band lower edges are inclusive	BVA	levelForScore(0.0) empty; levelForScore(0.5) = low	LevelBoundariesAreInclusiveAtTheLowerEdge	Pass
TC-RISK-006	Custom bands are honoured	Decision table	Configured bands produce medium	HonoursCustomBands	Pass
TC-SET-001	Shipped defaults are valid	Equivalence partitioning	validate() true, no error text	DefaultsAreValid	Pass
TC-SET-002	Zero large-amount threshold is rejected	BVA	Invalid; message names "Large amount"	RejectsZeroLargeAmountThreshold	Pass
TC-SET-003	Non-finite thresholds are rejected	Error guessing	NaN and infinity both invalid	RejectsNonFiniteThresholds	Pass
TC-SET-004	Zero duplicate tolerance is accepted	BVA	Valid — lower bound is closed at 0	AcceptsZeroDuplicateTolerance	Pass
TC-SET-005	Rapid withdrawal count below two is rejected	BVA	Invalid; message names "Rapid withdrawal count"	RejectsRapidWithdrawalCountBelowTwo	Pass
TC-SET-006	Zero-variance ratio at or below 1 is rejected	BVA	Invalid; message names "Zero variance ratio"	RejectsZeroVarianceRatioAtOrBelowOne	Pass
TC-SET-007	High threshold not above medium is rejected	Decision table	Invalid; message names "above the medium"	RejectsHighThresholdNotAboveMedium	Pass
TC-SET-008	High threshold above 100 is rejected	BVA	Invalid; message names "cannot exceed 100"	RejectsHighThresholdAboveHundred	Pass
TC-TXN-001	Zero and negative amounts are rejected	BVA	Rejected by repository validation	RejectsZeroAndNegativeAmounts	Pass
TC-TXN-002	Non-finite amounts are rejected	Error guessing	Rejected	RejectsNonFiniteAmounts	Pass

ID	Objective	Technique	Expected result	Test	Status
TC-TXN-003	Duplicate supplied transaction ID is rejected	Error guessing	Rejected; row not inserted	Rejects-ADuplicate-SuppliedId	Pass
TC-TXN-004	Account identifiers are normalised to canonical form	Equivalence partitioning	Stored in ACC + 4-character form	Normalises-Account-Identifiers	Pass
TC-CSV-001	Sample file imports completely	Equivalence partitioning	11 imported, 0 skipped, 0 failed, 11 rows in table	ImportsTheSample-File	Pass
TC-CSV-002	Re-importing the same file skips every row	State transition	0 imported, 11 skipped, table still 11	ReimportingThe-SameFileSkips-EveryRow	Pass
TC-CSV-003	Duplicate column is rejected	Error guessing	File rejected with an error	Rejects-ADuplicateColumn	Pass
TC-CSV-004	Blank lines are skipped, not failed	Error guessing	Valid rows imported, blank rows ignored	SkipsBlankLines	Pass
TC-INV-001	Resolving a case records its closing time	State transition	closed_at populated	ResolvingRecords-TheClosingTime	Pass
TC-INV-002	Reopening a case clears the closing time	State transition	closed_at cleared; history retained	ReopeningClears-TheClosingTime	Pass
TC-INV-003	Invalid status is rejected	Equivalence partitioning	Rejected by isValidStatus	RejectsAnInvalid-Status	Pass
TC-INV-004	Saving an unchanged case adds no event	State transition	Event count unchanged	SavingAn-UnchangedCase-AddsNoEvent	Pass
TC-SYS-001	Re-analysis does not duplicate anomaly rows	State transition	Row count stable across two runs	ReanalysisDoes-NotDuplicateRows	Pass
TC-SYS-002	Re-analysis preserves investigation work	State transition	Case assignments and notes survive	Reanalysis-Preserves-InvestigationWork	Pass
TC-SYS-003	Correcting a transaction removes its stale finding	State transition	Anomaly no longer present after correction	Correcting-ATransaction-RemovesTheStale-Finding	Pass
TC-TIME-001	Leap day parses correctly	BVA	29 Feb 2024 accepted	HandlesALeapDay	Pass
TC-TIME-002	Non-leap 29 February is rejected	BVA	Rejected as impossible	RejectsNonLeap-YearFebruary29	Pass
TC-TIME-003	Timestamps are unaffected by DST transitions	Error guessing	Same instant across a DST boundary	IsUnaffectedBy-DaylightSaving-Transitions	Pass

Summary: 45 designed cases, all implemented, all passing, distributed across eight suites. Every case has a test and every test has a case; neither list has an orphan.

C.3 Cases That Were Not Executed

Area	Case	Reason	Disposition
GUI rendering	Widget layout, table sorting, chart rendering	No widget harness; needs a display and a screenshot baseline	Manual acceptance only
ReportsPage	Any behaviour	Thirteen-line placeholder; no behaviour exists	Out of scope
SettingsPage	Any behaviour	Placeholder; no behaviour exists	Out of scope
Concurrency	Two clients writing simultaneously	Single-user local application; no network layer	Out of scope
Detection accuracy	Precision and recall against labelled fraud	No labelled ground-truth dataset exists	Not claimed anywhere
Scale beyond 20,000	Behaviour at 100,000+ transactions	Benchmark ceiling chosen for runtime	Extrapolated, not measured

Area	Case	Reason	Disposition
Portability	Behaviour on Windows or macOS	Tested on one Linux distribution	Declared limitation

The accuracy row is the one a reader is most likely to look for, so it is stated plainly: this catalogue verifies that each rule implements its specification. It does not and cannot verify that the specification detects real fraud.

C.4 What the Catalogue Demonstrates About the Design

Three patterns recur across the 46 cases, and they are the reason the catalogue is arranged by area rather than by suite.

Every threshold is pinned from both sides. `TC-LARGE-001/002/003` sit below, at, and above the large-amount limit; `TC-SET-005` pins the count boundary at two; `TC-RISK-005` pins the band edge at its inclusive lower end. A suite that only tested values above each threshold would pass an implementation that rejected everything.

State transitions are tested for reversal, not just progression. `TC-INV-001` closes a case and `TC-INV-002` reopens it, asserting that the closing time is cleared and that the history survives. Progression-only testing would miss an implementation that reopens a case by deleting its audit trail.

Idempotency is tested explicitly. `TC-SYS-001` and `TC-SYS-002` run detection twice and assert that nothing is duplicated and nothing is lost. This is the class of defect that unit tests structurally cannot reach, and it is where BUG-02 and BUG-03 were found.

Appendix D — Test Environment and Tooling

D.1 Tool inventory

#	Tool	Purpose	Version	Gate
1	GCC/G++	Compilation and diagnostics	14.2.0	build
2	GoogleTest	Unit and integration framework	1.16.0 (FetchContent, pinned)	X1
3	CTest	Test runner and reporting	CMake 3.31.6	X1
4	gcov	Structural coverage	14.2.0	X5
5	lcov	Coverage report generation	2.3.1-1	X5
6	Cppcheck	Static analysis	2.17.1	X4
7	Clang-Tidy	Static analysis and code quality	19.1.7	X4
8	AddressSanitizer + UBSan	Dynamic analysis	GCC 14.2.0	X3
9	Valgrind (Memcheck)	Dynamic analysis	3.24.0	X3
10	`ftad_benchmark`	Performance measurement	project-local	X8

All ten were executed on 4 November 2026. The lcov version string is inconsistent and is recorded rather than corrected: the Debian package is 2.3.1-1 according to `dpkg -s`, but the binary prints `LCOV version 2.0-1`. The package version is authoritative, and anyone reproducing the run will see the same output.

Per-gate commands and log files are listed in `documentation/evidence/README.md`; the consolidated result is `documentation/evidence/SUMMARY.md`.

D.2 Environment specification

Component	Value
Operating system	Parrot Security 7.4 (Debian 13 base)
Kernel	7.1.13+parrot7-amd64
Architecture	x86_64
Compiler	GCC/G++ 14.2.0 (Debian 14.2.0-19)

Component	Value
Build system	CMake 3.31.6, Ninja 1.12.1
GUI toolkit	Qt 6.8.2 (Widgets, Sql, Charts)
Database	SQLite 3.46.1 via Qt `QSQLITE`
Scripting	Python 3.13.5
C++ standard	C++17

The Release build used for test execution and benchmarking:

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release -DFTAD_BUILD_TESTS=ON
cmake --build build -j2
```

Coverage reconfigures the same tree with `-DFTAD_ENABLE_COVERAGE=ON`, adding `--coverage -O0 -g` to `ftad_core`. Sanitizer runs add `-DCMAKE_CXX_FLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer -g"`.

The environment is a rolling, security-oriented Linux distribution. It is representative of the target platform but is not a controlled benchmark platform: no CPU frequency scaling was pinned and no other processes were excluded during measurement. Timing figures in Chapter 6 must be read with that caveat.

Appendix E — Performance Measurement Record

E.1 Timestamp construction cost

The defect behind BUG-06. Each variant constructs the same date and time; only the time-zone handling differs. Measured over 20,000 records.

Construction	Time for 20,000 records
`QDateTime::fromString(ts, "yyyy-MM-dd HH:mm:ss")` (local)	2,917 ms
`QDateTime(date, time, Qt::LocalTime)`	2,901 ms
`QDateTime(date, time, Qt::UTC)`	6 ms

End-to-end analysis at the same size: 40.50 s before, 27.6 ms after — roughly 1,466×. The "before" case is not re-runnable once the defect is fixed, so it is attributed to its own run and is never blended with current figures.

E.2 Current baseline, 4 November 2026, Release `-O2`

Size	`analyze()`	Per transaction	`build()`
1,000	1.70 ms	1.70 µs	0.44 ms
5,000	9.94 ms	1.99 µs	3.13 ms
10,000	18.76 ms	1.88 µs	5.87 ms
20,000	39.88 ms	1.99 µs	12.87 ms

Scoring throughput is 0.238 µs per score. Each figure is the median of five repetitions.

E.3 Measurement noise

Three runs of the same binary on the same data gave 31.7 ms, 34.8 ms and 39.9 ms at 20,000 transactions — a spread of roughly 25 %. No CPU frequency scaling was pinned and other processes were running, so the host is not a controlled benchmark platform. The report quotes the range and the median (34.8 ms) and draws no conclusion from differences smaller than the spread.

E.4 Scaling

The measured scaling factor at 20,000 against 1,000 is 23.5×, against a linear ideal of 20×. The excess is consistent with cache effects and with the noise in E.3. The decisive observation is that per-transaction cost stays between 1.7 and 2.0 µs across the whole range: under quadratic behaviour that column would rise steadily with size, and it does not. The algorithm is therefore effectively linear, which is what the detection rules imply — each transaction is compared against a bounded history window rather than against all prior transactions.

Appendix F — Security Findings in Detail

F.1 SEC-02 and SEC-03 — no authentication, no encryption

There is no login, no user accounts and no roles: whoever can launch the application can read and modify everything, and anyone holding the database file can read it without the application at all. For a course project this is defensible, and the brief does not require authentication; what would not be defensible is silence about it. For real financial data both would be blocking, and neither is an incremental fix: authentication implies a user store, a session model and an authorisation policy on every repository method, while encryption at rest implies SQLCipher or a key-management story the application does not have. Both are recorded as accepted risks with a stated deployment constraint: FTAD suits single-user, single-machine use on data the operator already controls, and does not suit shared or regulated environments.

SEC-04 — database beside the executable. `DatabaseManager` resolves the database to `applicationDirPath() + "/database"`. Installed system-wide under `/opt` or `/usr/local`, the process either cannot write there — first launch fails for an ordinary user — or must run elevated, creating a root-owned world-readable database. The conventional location is `QStandardPaths::AppDataLocation`, which is per-user, platform-permissioned and survives reinstallation. This is a robustness finding with a security consequence.

SEC-05 — no tamper detection. Nothing detects direct edits to the database: an attacker with file access can change an amount, delete an anomaly or rewrite a note with `sqlite3`, and the application presents the result as its own. The schema enforces internal consistency — `amount >= 0`, `risk_score` between 0 and 100, `risk_level` restricted to three values, foreign keys enabled — but consistency is not authenticity, and detection requires a hash chain or signature, which is a substantial feature and is accepted rather than

recommended.

SEC-07 — CSV formula injection. Recorded because it is cheap to note now and expensive to discover later. `ReportsPage` exports nothing, so the risk is currently unreal. When export is implemented, a transaction field beginning with `=`, `+`, `-` or `@` will be interpreted as a formula by spreadsheet software, so anyone who can influence imported CSV content could plant a payload that executes when a reviewer opens the report; the mitigation is to prefix such fields with a single quote on export. The vulnerability does not exist yet, but the code path that will create it is already planned.

Appendix G — Architectural Views

The six views below were prepared as Mermaid sources under `documentation/diagrams/` and rendered for this report with Graphviz. They are reference material: Chapter 3 states the structure in prose, and these figures show it. Each was checked against the module list in §3.3 and the schema in `database/schema.sql`; where a page is a placeholder it is marked with an asterisk.

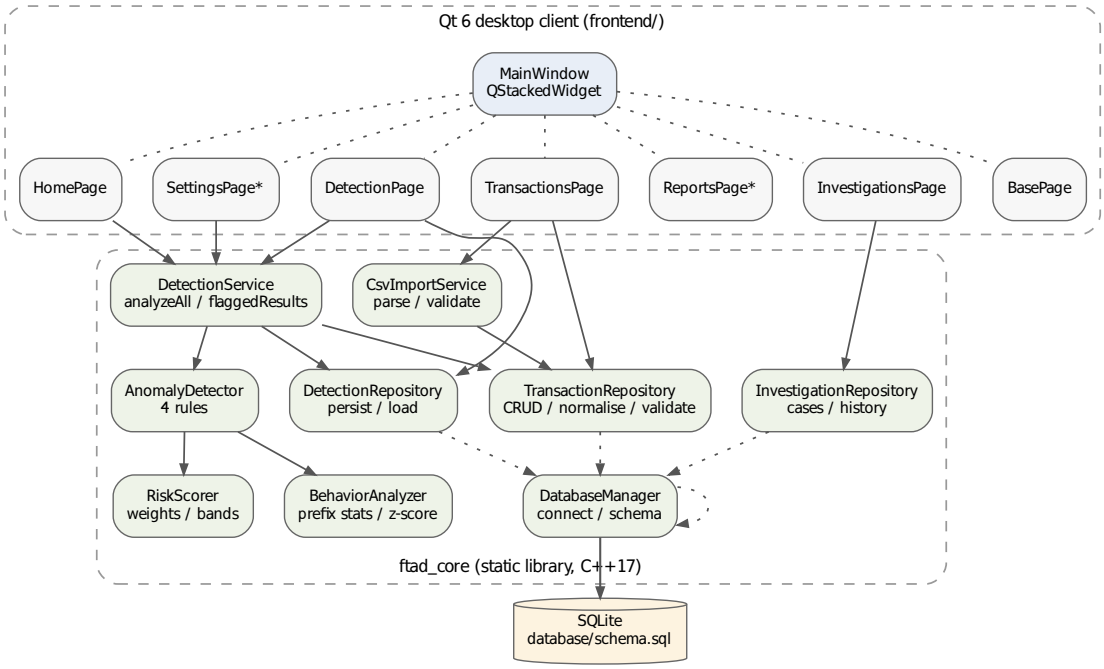


Figure 1: Figure G.1 — Layering and call relationships. The Qt client sits above the `ftad_core` static library, which is the only component that touches SQLite. `ReportsPage` and `SettingsPage` (asterisked) are placeholders.

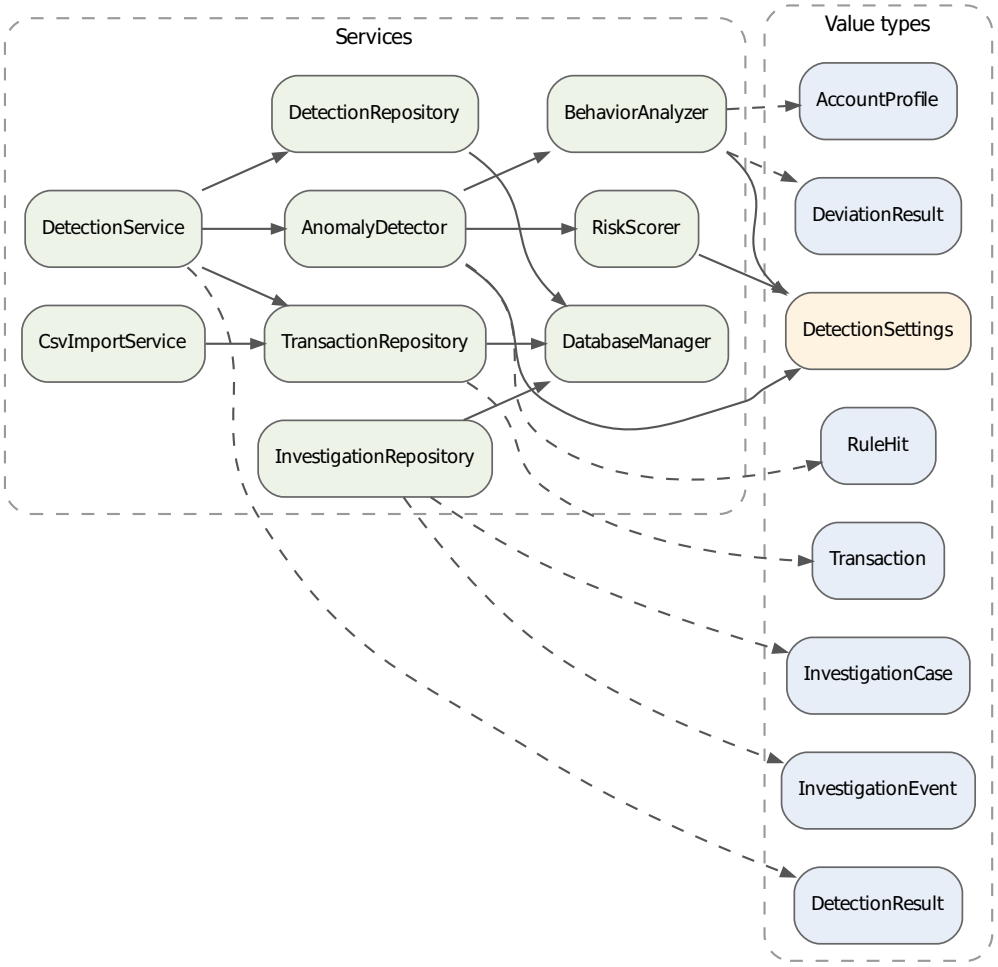


Figure 2: Figure G.2 — Backend classes. Services on the left collaborate through the value types on the right; `DetectionSettings` (shaded) is the shared configuration object that every rule reads.

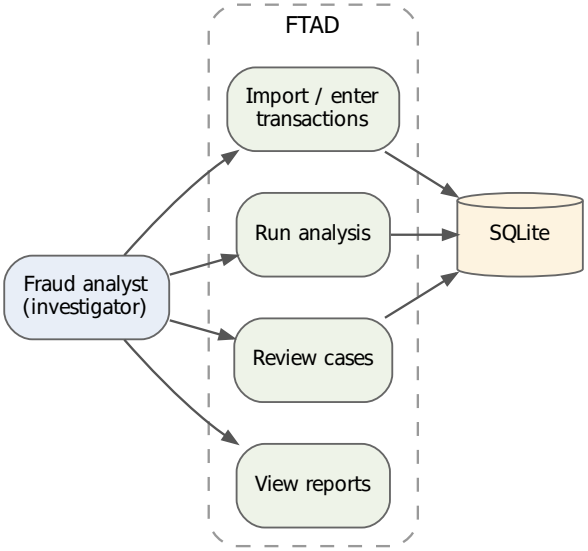


Figure 3: Figure G.3 — Actors and use cases. A single analyst role drives import, analysis, case review and reporting, all against the local database.



Figure 4: Figure G.4 — Detection activity. Every transaction passes all four rule checks before scoring, so a single transaction can accumulate several hits.



Figure 5: Figure G.5 — Call sequence for one analysis pass, from the UI through `DetectionService` to persistence and back.

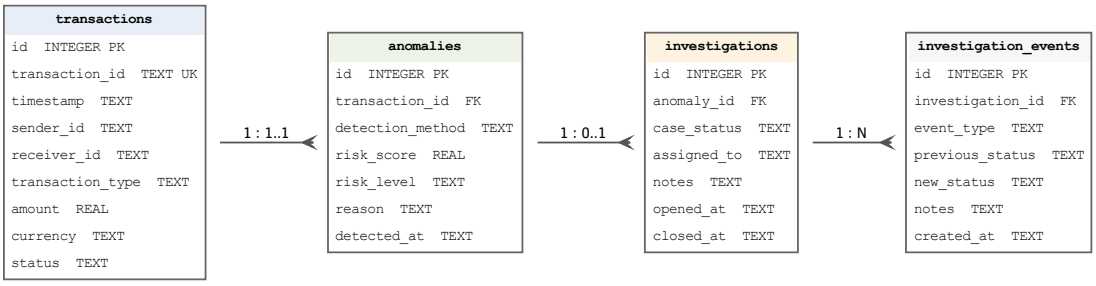


Figure 6: Figure G.6 — Entity relationships. Foreign keys run transactions ☐ anomalies ☐ investigations ☐ investigation_events, with cascade deletes in the same direction.